

CORE-MP: Computación Objetiva de Relaciones Exitosas mediante Modelos Predictivos. Un enfoque data-driven para la evaluación de compatibilidad.

Raúl Bernalte Sánchez

José Manuel Bastante Rodríguez

Óscar Domínguez Ocaña

José Alberto Seco Sánchez-Camacho

Dr. José Antonio De La Torre Las Heras

Dr. Julio Alberto López Gómez

Introducción	4
1 Definición del problema y determinación del alcance	6
1.1 Contexto y definición del problema	6
1.2 Objetivos generales y específicos	8
1.3 Determinación del alcance	9
1.3.1 Planificación	9
1.3.2 Recursos Humanos: asignación de tareas y capacidad	10
1.3.3 Delimitación funcional del sistema	11
1.3.4 Viabilidad y estimación del ROI	12
1.3.5 Valor	13
2 Los datos: origen, descripción y análisis	14
2.1 Introducción	14
2.2 Fuentes, origen y tamaño de los datos	14
2.3 Inventario de datos	15
2.4 Análisis de los datos y Detección de Sesgos	18
2.4.1 Análisis de Sesgos: Variables Objetivo (Score, Clases y Longevidad)	18
2.4.2 Análisis de Sesgos Estructurales: Ambición y Brecha de Edad	20
2.4.3 Análisis de Sesgos de Interacción: Lenguajes del Amor	20
2.4.4 Ingeniería de Características (<i>Feature Engineering</i>)	21
2.5 Evaluación del rendimiento y Métricas	22
2.5.1 1. Predicción de Compatibilidad Binaria	22
2.5.2 2. Estimación de Longevidad Relacional	23
2.5.3 3. Recomendación (Matching óptimo)	23

3	Modelado, Experimentación y Validación	25
3.1	Introducción	25
3.2	Diseño Experimental	25
3.2.1	Partición de Datos	25
3.2.2	Selección de Características	26
3.2.3	Modelos Evaluados	26
3.2.4	Métricas de Evaluación	28
3.3	Resultados de Clasificación: Predicción de Compatibilidad	29
3.3.1	Análisis de Resultados de Clasificación	29
3.3.2	Importancia relativa de variables por modelo en clasificación	31
3.3.3	Evaluación Final del Modelo Seleccionado: Regresión Logística	32
3.3.4	Interpretabilidad: Coeficientes del Modelo	32
3.4	Resultados de Regresión: Estimación de Longevidad	33
3.4.1	Análisis de Resultados de Regresión	33
3.4.2	Importancia relativa de variables por modelo en regresión	34
3.4.3	Evaluación Final del Modelo Seleccionado: Regresión Ridge	34
3.5	Discusión	35
3.5.1	Comparativa entre Clasificación y Regresión	35
3.5.2	Elección de Modelos Simples frente a Modelos Complejos	35
3.6	Conclusiones	36
4	Despliegue y Puesta en Producción	37
4.1	Introducción	37
4.2	Arquitectura del Sistema	38
4.2.1	Visión General	38
4.2.2	Justificación del Diseño	38
4.3	Pipeline de Ingeniería de Características en Producción	39
4.3.1	El Principio Stateless / Stateful	39
4.4	Implementación de la API REST	39
4.4.1	Framework y Tecnología	39
4.4.2	Gestión del Ciclo de Vida	40
4.4.3	Endpoints Disponibles	40
4.4.4	Esquema de Entrada y Salida	40
4.5	Containerización con Docker	41
4.5.1	Imágenes Docker	41
4.5.2	Orquestación con Docker Compose	41

4.6	Estructura Modular del Código	41
4.6.1	Organización por Dominio Funcional	41
4.7	Estrategia de Monitoreo	42
4.7.1	Métricas de Latencia	42
4.7.2	Comprobación de Estado (<i>Health Check</i>)	42
4.8	Conclusiones	42
5	Monitorización y Feedback Loop	44
5.1	Introducción	44
5.2	Arquitectura de Monitorización	44
5.3	Métricas Operativas y Dashboard	45
5.4	Detección de Drift de Datos	46
5.5	Feedback Loop con Airflow y Alertas	47
5.6	Ciclo de Vida del Modelo	48
5.7	Conclusiones	49
	Conclusiones	50

List of Tables

2.1	Inventario de datos original: Diccionario de variables	16
2.2	Diccionario de variables interactivas: características con pesos positivos y negativos	22
3.1	Configuración de los modelos evaluados para la tarea de clasificación binaria (compatible).	27
3.2	Configuración de los modelos evaluados para la tarea de regresión (relationship_longevity_months).	28
3.3	Métricas de clasificación sobre el conjunto de validación (15.000 muestras). La métrica principal de selección de modelo es el F1-Score.	29
3.4	Comparación del rendimiento de la Regresión Logística en validación y test. La mejora marginal en test indica ausencia de sobreajuste.	32
3.5	Métricas de regresión sobre el conjunto de validación. La métrica principal de selección es el MAE.	33
3.6	Comparación del rendimiento de la Regresión Ridge en validación y test.	35
4.1	Endpoints de la API REST del sistema CORE-MP	40
5.1	Contenedores del stack de monitorización de CORE-MP	44
5.2	Paneles del dashboard de Grafana para el sistema CORE-MP	46

List of Figures

2.1	Extracto de los datos del Individuo A correspondientes a los 24 primeros emparejamientos evaluados. Se aprecian las variables demográficas y las métricas del modelo <i>Big Five</i>	17
2.2	Extracto de los datos del Individuo B correspondientes a los 24 primeros emparejamientos. Nótese la obligatoria correspondencia del campo <code>pair_id</code> con el Individuo A para posibilitar el emparejamiento.	17
2.3	Representación de las variables objetivo que definen el nivel de satisfacción y viabilidad de la relación, correspondientes a los 24 primeros registros del conjunto de datos.	18
2.4	Distribución de las variables objetivo: densidad del score (izq.), balance de clases (centro) y longevidad (der.).	19
2.5	Distribución de registros según su longevidad agrupada en intervalos de 5 meses. Se aprecia la fuerte caída temprana de las relaciones incompatibles.	19
2.6	Análisis de sesgos estructurales: Afinidad en ambición profesional (izquierda) y diagrama de violín para la diferencia de edad (derecha).	20
2.7	Matriz de contingencia térmica (<i>Heatmap</i>): Tasa de compatibilidad porcentual según el cruce de los lenguajes del amor entre ambos individuos.	21
3.1	Matriz de confusión en validación	30
3.2	Matriz de confusión en test	31
3.3	Importancia relativa de las variables en los modelos de clasificación. Los valores están normalizados por modelo y permiten comparar la relevancia de cada característica en la predicción de la compatibilidad.	32
3.4	Importancia relativa de las variables en los modelos de regresión. Los valores están normalizados por modelo y muestran una distribución más homogénea de la relevancia, sin variables claramente dominantes.	34

4.1	Arquitectura de dos contenedores del sistema CORE-MP. El volumen compartido actúa como contrato entre el pipeline de entrenamiento y el servicio de inferencia.	38
5.1	Flujo de datos del stack de monitorización. Flechas sólidas: métricas; discontinuas: acciones de control del feedback loop.	45
5.2	Dashboard de Grafana con los cuatro paneles de monitorización operativa de CORE-MP en tiempo real.	46
5.3	Reglas de alerta en Grafana. Izquierda: “API caída” (consulta <code>up{job="core_mp_api"}</code> , umbral <i>below</i> 1). Derecha: “Latencia P95 alta” (<code>histogram_quantile(0.95, ...)</code> , umbral <i>above</i> 0.5 s). Ambas notifican vía <i>contact point</i> telegram-g5.	47
5.4	Pantalla de <i>Contact points</i> de Grafana: el canal telegram-g5 aparece asociado a 1 política de notificación y 2 reglas de alerta, con última entrega hace 2 minutos.	48
5.5	Ciclo completo de alerta vía Telegram: mensaje FIRING (izquierda) al caer la API y RESOLVED (derecha) al recuperarse, 1 min después.	48
5.6	Ciclo de vida continuo del modelo CORE-MP: las predicciones alimentan la detección de drift, que activa el reentrenamiento y la recarga sin interrumpir el servicio.	49

1

En el contexto de la **transformación digital de las relaciones interpersonales**, las plataformas de citas y aplicaciones de *matching* han adquirido un papel central en la forma en que las personas establecen conexiones afectivas. En la actualidad, estas plataformas gestionan enormes volúmenes de información sobre preferencias, rasgos de personalidad, características demográficas y patrones de comportamiento. El objetivo fundamental de este ecosistema digital es procesar dicha información para facilitar la formación de **relaciones compatibles y potencialmente duraderas**, actuando como intermediarios algorítmicos en el tejido social.

Tradicionalmente, los sistemas de recomendación en este ámbito se han basado en filtros simples, reglas heurísticas o mecanismos poco transparentes que priorizan coincidencias superficiales (como la edad, la localización geográfica o los intereses básicos). Sin embargo, la compatibilidad relacional es un **fenómeno inherentemente complejo y multidimensional**. El éxito de una relación no se define por la simple superposición de aficiones, sino que depende de una intrincada red de factores psicológicos, valores personales, estilo de vida, ambiciones profesionales y patrones emocionales. La incapacidad de los sistemas tradicionales para capturar esta profundidad suele derivar en recomendaciones irrelevantes, generando frustración y fatiga en el usuario.

En este contexto, surge la necesidad crítica de diseñar **sistemas de inteligencia artificial** capaces de modelar de forma cuantitativa y estructurada la compatibilidad entre individuos. Al integrar múltiples dimensiones personales, estos sistemas pueden descubrir patrones latentes y generar predicciones que sean **consistentes, reproducibles y explicables**. En el delicado ámbito de las relaciones humanas, un sistema de este tipo no solo debe alcanzar un alto rendimiento técnico o estadístico, sino también garantizar su interpretabilidad. Resulta imperativo mantener la coherencia con los principios del dominio relacional, evitando a toda costa funcionar como una "caja negra" que tome decisiones opacas o arbitrarias.

El presente proyecto se fundamenta en el dataset "*Cupid's Algorithm*" [**dataset_cupid**], un conjunto de datos que contiene información estructurada y enriquecida sobre pares de individuos. Este *dataset* abarca

¹El formato y la estructura documental de esta memoria han sido desarrollados utilizando la plantilla oficial de proyectos de la ULB [**ulb_template**].

desde variables demográficas y profesionales hasta métricas psicométricas profundas, como los rasgos de personalidad basados en el modelo **Big Five** [rakic2024bigfive] y los lenguajes del amor, acompañados de indicadores históricos de compatibilidad y duración potencial de la relación. La riqueza de esta información permite abordar el reto desde una **perspectiva supervisada**, formalizando la afinidad relacional como una variable matemática susceptible de ser modelada mediante técnicas avanzadas de aprendizaje automático.

A partir de este marco conceptual, el proyecto no se limita al mero entrenamiento de un modelo predictivo aislado en un entorno académico, sino que adopta una visión integral del **sistema de inteligencia artificial como producto**. Esto implica recorrer, de manera rigurosa, todas las fases del ciclo de vida de un sistema de *Machine Learning* (MLOps): desde la definición formal del problema y la preparación de los datos, pasando por el modelado y la evaluación, hasta culminar en su **integración como servicio de software** y su correspondiente análisis de viabilidad técnica y económica.

De este modo, el objetivo principal del proyecto es diseñar e implementar un **sistema inteligente** capaz de predecir la compatibilidad entre dos individuos de forma completamente objetiva y explicable. Un sistema concebido desde su inicio para ser un componente integrable en una plataforma digital de relaciones, y evaluado no solo a través de métricas de precisión técnica, sino también por su **potencial impacto organizativo, económico y social**.

Definición del problema y determinación del alcance

En este capítulo se detalla la definición del problema seleccionado para la evaluación del proyecto de la asignatura Desarrollo e Integración de Servicios de IA así como la determinación de su alcance.

1.1 Contexto y definición del problema

En el ámbito de las plataformas digitales de relaciones interpersonales, la predicción de compatibilidad entre individuos constituye un problema de alta relevancia tecnológica, social y económica. Las aplicaciones de citas modernas gestionan grandes volúmenes de información estructurada sobre los usuarios, incluyendo variables demográficas, profesionales, rasgos de personalidad y preferencias relacionales. A partir de estos datos, los sistemas deben decidir qué perfiles recomendar y qué emparejamientos priorizar.

La compatibilidad relacional es, sin embargo, un fenómeno inherentemente complejo y multidimensional. No depende exclusivamente de variables superficiales como la edad o la localización geográfica, sino que está influida por la interacción entre rasgos de personalidad, valores personales, ambiciones profesionales, estilos de comunicación, hábitos cotidianos y expectativas afectivas. Esta naturaleza multifactorial convierte la modelización de la compatibilidad en un problema analítico no trivial.

Tradicionalmente, muchas plataformas han empleado reglas heurísticas o sistemas de filtrado simples basados en coincidencias directas (por ejemplo, misma localización o intereses compartidos). Estos enfoques presentan limitaciones evidentes: no capturan interacciones complejas entre variables, no permiten aprender patrones implícitos presentes en los datos y suelen carecer de transparencia en la forma en que generan recomendaciones.

El dataset seleccionado para este proyecto, “Cupid’s Algorithm”, contiene información estructurada sobre pares de individuos (Person A – Person B), incluyendo:

- Variables demográficas (edad, nivel educativo, localización).

- Variables profesionales (campo profesional, nivel de ambición).
- Rasgos de personalidad basados en el modelo Big Five.
- Variables relacionales (lenguaje del amor, cronotipo, espontaneidad, expresividad emocional).
- Una variable continua de compatibilidad (*compatibility_score*).
- Una variable binaria de compatibilidad (*compatible*).
- Un indicador de longevidad relacional (*relationship_longe*).

La presencia de estas variables permite abordar el problema desde una perspectiva supervisada, donde la compatibilidad observada actúa como señal objetivo para el aprendizaje.

Desde el punto de vista formal, el problema puede plantearse como un problema de clasificación binaria. Cada par de individuos queda caracterizado por un conjunto de variables numéricas y categóricas que describen sus atributos individuales. El sistema debe aprender una función:

$$f(X_A, X_B) = P(\text{compatible} = 1)$$

Donde X_A y X_B representan los vectores de características (demográficas y psicológicas) de los individuos A y B.

Alternativamente, el problema puede formularse como una regresión cuando se utiliza la variable continua *compatibility_score* como objetivo, o como una predicción de duración cuando se emplea *relationship_longe*. No obstante, en este proyecto se delimita el alcance principal al problema de clasificación binaria, al tratarse de una formulación clara, evaluable y directamente alineada con aplicaciones reales de matching.

El sistema no debe limitarse a generar una predicción numérica, sino que debe concebirse como un sistema de inteligencia artificial integrable en un entorno software real. Esto implica que el modelo debe ser reproducible, evaluable mediante métricas adecuadas y potencialmente explicable, de manera que las decisiones puedan ser comprendidas por responsables de producto y desarrolladores de la plataforma.

En consecuencia, el problema queda claramente delimitado: a partir de datos estructurados que describen características personales, profesionales y psicológicas de dos individuos, se busca diseñar un sistema de inteligencia artificial capaz de predecir de forma objetiva, consistente y explicable su compatibilidad relacional, concebido como un componente integrable dentro de una arquitectura software más amplia.

1.2 Objetivos generales y específicos

El problema que se aborda en este proyecto puede formularse de la siguiente manera: ¿Cómo diseñar un sistema de inteligencia artificial capaz de predecir de forma objetiva, consistente y explicable la compatibilidad entre dos individuos, a partir de variables demográficas, profesionales y de personalidad, garantizando su integración como servicio software en una plataforma digital de relaciones?

Esta formulación no se limita a la obtención de buenas métricas técnicas, sino que incorpora explícitamente la dimensión de sistema IA como producto, concebido para integrarse en un entorno real, evolucionar en el tiempo y aportar valor a distintos stakeholders.

Para dar respuesta a esta pregunta, se establece como objetivo general del proyecto diseñar, implementar y validar un sistema de inteligencia artificial capaz de predecir la compatibilidad relacional entre dos individuos utilizando datos estructurados, garantizando su integración en una arquitectura software real y su funcionamiento fiable, explicable y reproducible a lo largo del tiempo.

Para alcanzar el objetivo general propuesto, se establecen los siguientes objetivos específicos:

- Analizar y formalizar el concepto de compatibilidad relacional desde una perspectiva cuantitativa y multidimensional, identificando las variables más relevantes en el dataset seleccionado.
- Seleccionar y procesar los datos del dataset Cupid's Algorithm, incluyendo limpieza, codificación de variables categóricas y construcción de variables derivadas que representen similitud o distancia entre perfiles (por ejemplo, diferencias de edad o distancias en rasgos de personalidad).
- Diseñar y entrenar modelos de aprendizaje automático supervisado que permitan predecir la variable binaria compatible, evaluando distintas aproximaciones metodológicas.
- Evaluar el rendimiento de los modelos mediante métricas técnicas adecuadas para problemas de clasificación binaria (Accuracy, Precision, Recall, F1-score, ROC-AUC), así como analizar su comportamiento desde una perspectiva alineada con el dominio.
- Incorporar mecanismos de explicabilidad, que permitan interpretar la influencia relativa de las distintas variables en la predicción del modelo y facilitar su comprensión por parte de perfiles no técnicos.
- Diseñar el sistema como un servicio integrable, desarrollando una API REST que permita realizar predicciones de compatibilidad a partir de nuevos pares de individuos.
- Analizar la viabilidad económica y organizativa del sistema, estimando costes de desarrollo y potencial retorno de la inversión (ROI) en un escenario realista de aplicación.

- Plantear posibles líneas de evolución futura, como la extensión hacia sistemas de recomendación top-K o la incorporación de modelos de predicción de duración relacional.

De este modo, el proyecto no se concibe como un simple ejercicio de modelado predictivo, sino como el desarrollo completo de un sistema de inteligencia artificial entendido como producto tecnológico, recorriendo todas las fases de su ciclo de vida: definición del problema, preparación de datos, modelado, evaluación, integración y análisis de valor.

1.3 Determinación del alcance

En esta sección se define el alcance del proyecto a desarrollar en la asignatura, prestando especial atención a los componentes clave que delimitan un sistema de inteligencia artificial concebido como producto: planificación temporal, organización del equipo, viabilidad económica y valor generado.

El alcance no solo determina qué se va a desarrollar, sino también qué queda explícitamente fuera del proyecto, con el fin de garantizar su viabilidad dentro del marco temporal de la asignatura y evitar una expansión descontrolada de requisitos.

1.3.1 Planificación

El proyecto se desarrollará en un periodo estimado de ocho semanas, estructurado en fases alineadas con el ciclo de vida de un sistema de Machine Learning:

Fase 1 – Análisis y comprensión del problema (Semanas 1–2)

- Estudio detallado del dataset Cupid's Algorithm.
- Análisis exploratorio de variables.
- Formalización definitiva del problema de clasificación binaria.

Fase 2 – Preparación y transformación de datos (Semanas 3–4)

- Limpieza de datos.
- Codificación de variables categóricas.
- Construcción de variables derivadas (similitudes, diferencias y distancias entre perfiles).
- División en conjuntos de entrenamiento y validación.

Fase 3 – Modelado y evaluación (Semanas 5–6)

- Entrenamiento de distintos modelos supervisados.

- Comparación de métricas.
- Selección del modelo final.
- Análisis de explicabilidad.

Fase 4 – Integración como servicio (Semana 7)

- Serialización del modelo entrenado.
- Desarrollo de API REST para predicción.
- Pruebas funcionales del sistema.

Fase 5 – Validación y análisis de valor (Semana 8)

- Evaluación final del sistema.
- Análisis de viabilidad y ROI.
- Redacción de documentación final.

Esta planificación permite recorrer todas las fases fundamentales del desarrollo de un sistema IA como producto sin exceder el marco temporal de la asignatura.

1.3.2 Recursos Humanos: asignación de tareas y capacidad

El desarrollo del sistema CUPID-AI se estructura como un proyecto profesional de inteligencia artificial, con un equipo técnico compuesto por cuatro miembros, cada uno con responsabilidades diferenciadas y complementarias. Equipo técnico de desarrollo 1. Experto en Dominio y Analista de Resultados (Domain Expert / AI Analyst) Responsable de:

- Formalizar el concepto de compatibilidad relacional.
- Analizar la coherencia de los resultados del modelo.
- Evaluar la interpretabilidad del sistema.
- Validar las predicciones desde una perspectiva conceptual y de producto.

Este rol actúa como enlace entre el modelo matemático y la interpretación relacional del problema, asegurando que las predicciones sean coherentes y justificables.

2. Analista de Datos (Data Analyst / Data Engineer) Responsable de:

- Exploración y limpieza del dataset.
- Transformación de variables.

- Construcción de métricas de similitud entre perfiles.
- Garantizar la calidad y consistencia del conjunto de datos.

Este rol constituye la base del sistema, ya que la calidad del modelo depende directamente de la calidad de los datos de entrada.

3. Ingeniero de Machine Learning (ML Engineer / AI Researcher) Responsable de:

- Selección y diseño de modelos supervisados.
- Ajuste de hiperparámetros.
- Evaluación comparativa de modelos.
- Implementación de métricas técnicas.

Este rol conecta con la fase de modelado, evaluación técnica y validación cuantitativa del sistema.

4. Ingeniero de Integración y MLOps (ML Systems Engineer) Responsable de:

- Serialización y versionado del modelo entrenado.
- Desarrollo de API REST para predicción.
- Diseño de arquitectura del sistema.
- Pruebas de integración y funcionamiento.

Este perfil garantiza que el proyecto no se limite a un modelo en un notebook, sino que se materialice como un servicio integrable dentro de una plataforma digital.

Esta estructura organizativa replica un escenario realista de desarrollo de sistemas de inteligencia artificial, donde intervienen perfiles técnicos especializados en datos, modelado e integración, así como un perfil orientado al análisis conceptual y validación.

1.3.3 Delimitación funcional del sistema

El alcance funcional del proyecto se limita a:

- Predicción binaria de compatibilidad (compatible).
- Uso exclusivo del dataset Cupid's Algorithm.
- Desarrollo de una API REST básica para predicción.
- Evaluación offline mediante métricas estándar.

Quedan explícitamente fuera del alcance:

- Desarrollo de una plataforma web completa.
- Implementación de un sistema de recomendación top-K a gran escala.
- Integración con bases de datos reales de usuarios.
- Despliegue en entorno cloud productivo.

Esta delimitación garantiza que el proyecto sea viable en el marco temporal disponible.

1.3.4 Viabilidad y estimación del ROI

Para estimar la viabilidad económica del sistema se plantean los siguientes supuestos:

- Equipo técnico: 4 personas.
- Dedicación media: 8 horas semanales por persona.
- Duración: 8 semanas.
- Coste medio por hora: 40 €.

Horas totales estimadas:

$$\text{Horas totales} = 4 \text{ personas} \times 8 \text{ h/semana} \times 8 \text{ semanas} = 256 \text{ horas}$$

Coste total estimado:

$$256 \text{ horas} \times 40 \text{ €/hora} = 10.240 \text{ €}$$

Desde el punto de vista organizativo, un sistema de predicción de compatibilidad puede generar beneficios en forma de:

- Incremento de la tasa de emparejamientos exitosos.
- Reducción del abandono de usuarios.
- Mejora de la satisfacción y fidelización.
- Posible comercialización del motor de matching como servicio.

Si el sistema generara un beneficio acumulado conservador de 15.000 € en un periodo de uso prolongado, el retorno de la inversión sería:

$$ROI = \frac{\text{Beneficio} - \text{Coste}}{\text{Coste}} = \frac{15000 - 10240}{10240} = 0.46$$

Es decir, un 46% de retorno estimado, lo que indica viabilidad económica.

1.3.5 Valor

El valor del sistema CUPID-AI se analiza desde múltiples dimensiones:

Valor tecnológico:

- Formalización de la compatibilidad como problema supervisado.
- Modelo reproducible y explicable.
- Arquitectura integrable como microservicio.

Valor organizativo:

- Reutilizable y escalable.
- Base para sistemas más avanzados de recomendación.

Valor social:

- Mayor transparencia en el proceso de matching.
- Reducción de decisiones arbitrarias.
- Mejora de confianza del usuario.

Con esta delimitación, el proyecto se concibe como el desarrollo completo de un sistema de inteligencia artificial entendido como producto tecnológico, cubriendo todas las fases de su ciclo de vida y garantizando su viabilidad dentro del marco temporal y académico de la asignatura.

Los datos: origen, descripción y análisis

2.1 Introducción

En este capítulo se detalla el conjunto de datos utilizado para el desarrollo e integración del sistema de Inteligencia Artificial del proyecto *Cupid Algorithm*. A diferencia de los problemas de clasificación tradicional, donde se perfila a un individuo de forma aislada, el presente reto analítico requiere el estudio de una **diada** (la pareja concebida como unidad matemática).

El objetivo principal de esta sección es comprender cómo interactúan y se correlacionan los perfiles psicológicos y demográficos, establecer las métricas formales de éxito y definir los procesos de transformación de datos (*Feature Engineering*) necesarios para alimentar el modelo predictivo. Durante el desarrollo del análisis se abordará el debate psicológico subyacente sobre si la afinidad relacional se fundamenta en la complementariedad (polos opuestos) o en la homofilia (similitud), justificando empíricamente las decisiones algorítmicas adoptadas.

2.2 Fuentes, origen y tamaño de los datos

La fuente de información para realizar esta investigación es `cupid_algorithm_dataset.csv`. Este *dataset* emula el entorno operativo de una aplicación de emparejamiento (*matchmaking*) a gran escala, registrando interacciones reales entre usuarios.

En términos de volumetría, la muestra consta de **100.000 observaciones** (pares evaluados) y 30 características originales. La magnitud y completitud del conjunto de datos garantizan la representatividad estadística necesaria para entrenar modelos de aprendizaje automático de alta complejidad, mitigando el riesgo de sobreajuste (*overfitting*) temprano. Asimismo, la validación de calidad del dato evidenció una estructura robusta: la inspección inicial confirmó la ausencia total de valores nulos o faltantes, lo cual permite prescindir de técnicas de imputación sintética que pudieran introducir sesgos artificiales en las distribuciones originales.

2.3 Inventario de datos

La base de datos presenta una arquitectura simétrica, registrando idéntica batería de atributos tanto para el “Individuo A” como para el “Individuo B” de cada emparejamiento. Estas variables se agrupan formalmente en las siguientes dimensiones:

- **Dimensión Demográfica y Sociológica:** Contiene atributos descriptivos tales como la edad, el nivel educativo (modelado como variable ordinal), la ubicación geográfica o entorno de residencia (*location*) y el campo profesional o sector productivo al que pertenecen (*career_field*).
- **Dimensión Psicológica y Conductual:** Agrupa valores normalizados en el rango [0, 1] que cuantifican la ambición profesional, la expresividad emocional, la espontaneidad y el cronotipo de cada sujeto. Resulta especialmente relevante la inclusión del modelo de los Cinco Grandes (*Big Five*): apertura a la experiencia (*openness*), extraversión, amabilidad (*agreeableness*) y responsabilidad (*conscientiousness*). Adicionalmente, se tipifica el estilo afectivo de los individuos mediante su lenguaje del amor principal.
- **Variables Objetivo (*Targets*):** Representan la verdad terreno (*ground truth*) con la que se evaluará el desempeño del sistema:
 - *compatibility_score*: Puntuación continua que evalúa la calidad y viabilidad general del emparejamiento (problema de regresión).
 - *compatible*: Etiqueta binaria de clasificación donde se codifica el éxito (1) o el fracaso (0) de la relación.
 - *relationship_longevity_months*: Duración efectiva de la relación expresada en meses.

Para facilitar la comprensión global de la estructura base del sistema, la Tabla 2.1 detalla el diccionario de datos original. Se listan las variables de manera genérica, asumiendo que el modelo procesa las características tanto del individuo A (*a_**) como del individuo B (*b_**).

Variable	Tipo	Descripción
pair_id	Entero	Identificador único (1 a 100,000) de cada pareja evaluada. Actúa como clave primaria.
age	Numérica	Edad cronológica (18-55). Se emplea para calcular la brecha de edad; diferencias superiores a 15 años penalizan el <i>score</i> .
education	Ordinal	Nivel educativo (1: Secundaria - 5: Doctorado). Correlaciona positivamente con la ambición profesional.
location	Categórica	Entorno de residencia (Urbano, Suburbano, Rural). Las coincidencias suman positivamente al evaluar la proximidad.
career_field	Categórica	Sector profesional (10 industrias). Su compatibilidad cruzada se evalúa internamente mediante una matriz de sinergia.
career_ambition	Num. [0,1]	Impulso de crecimiento profesional. La proximidad de valores entre la pareja indica alineación de proyectos de vida.
openness	Num. [0,1]	Rasgo <i>Big Five</i> : apertura a la experiencia, ideas nuevas y creatividad.
extraversion	Num. [0,1]	Rasgo <i>Big Five</i> : nivel de extroversión, sociabilidad, energía y asertividad social.
agreeableness	Num. [0,1]	Rasgo <i>Big Five</i> : amabilidad, tendencia a la compasión, empatía y voluntad de compromiso.
conscientiousness	Num. [0,1]	Rasgo <i>Big Five</i> : nivel de organización, disciplina y responsabilidad personal.
chronotype	Num. [0,1]	Preferencia de estilo de vida que abarca desde perfiles nocturnos hasta matutinos.
spontaneity	Num. [0,1]	Orientación vital entre la planificación estructurada (0) y el espíritu libre (1).
emotional_express...	Num. [0,1]	Estilo de comunicación: facilidad y franqueza al expresar y compartir sentimientos íntimos.
love_language	Categórica	Lenguaje del amor principal (Palabras de afirmación, Actos de servicio, Regalos, Tiempo de calidad o Contacto físico).
compatibility_score	Continua	Target: Puntuación predictiva agregada (0-100) que determina la compatibilidad base de la pareja.
compatible	Binaria	Target: Éxito relacional (1) o fracaso (0). Umbral original establecido en >65 de <i>score</i> .
relationship_lon...	Entero	Target: Longevidad de la relación expresada en meses (0-120). Útil para problemas de regresión o supervivencia.

Table 2.1: Inventario de datos original: Diccionario de variables

Para ilustrar de forma tangible esta arquitectura simétrica, las Figuras 2.1 y 2.2 muestran, de manera independiente, los perfiles de los usuarios vinculados que conforman los primeros 24 registros del conjunto de datos. Como se puede observar, ambos sujetos (Individuo A e Individuo B) mantienen de forma estricta el mismo identificador de pareja (pair_id). Esta restricción relacional es la que permite al sistema procesar la interacción de ambos como una unidad indisoluble a la hora de realizar la inferencia.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	pair_id	a_age	a_education	a_location	a_career_field	a_career_ambition	a_openness	a_extraversion	a_agreeableness	a_conscientiousness	a_chronotype	a_spontaneity	a_love_language	a_emotional_expressiveness
2	1	46	3	Suburban	Healthcare	0.23	0.67	0.78	0.32	0.49	0.02	0.38	Receiving Gifts	0.59
3	2	32	2	Suburban	Tech	0.58	0.78	0.7	0.51	0.71	0.43	0.72	Quality Time	0.73
4	3	25	4	Rural	Marketing	0.59	0.33	0.87	0.64	0.82	0.5	0.57	Acts of Service	0.69
5	4	38	4	Suburban	Finance	0.54	0.34	0.28	0.72	0.81	0.43	0.6	Physical Touch	0.47
6	5	36	2	Rural	Entrepreneurship	0.56	0.35	0.62	0.27	0.73	0.32	0.63	Acts of Service	0.44
7	6	40	3	Urban	Science	0.75	0.45	0.4	0.71	0.54	0.26	0.24	Words of Affirmation	0.77
8	7	28	1	Suburban	Tech	0.36	0.47	0.84	0.56	0.49	0.61	0.91	Quality Time	0.76
9	8	28	1	Suburban	Engineering	0.0	0.28	0.33	0.95	0.76	0.18	0.15	Quality Time	0.47
10	9	41	4	Urban	Entrepreneurship	0.52	0.69	0.5	0.25	0.34	0.15	0.59	Words of Affirmation	0.33
11	10	53	5	Rural	Law	0.75	0.65	0.61	0.49	0.28	0.41	0.51	Words of Affirmation	0.71
12	11	41	2	Urban	Finance	0.39	0.15	0.46	0.85	0.89	0.52	0.02	Words of Affirmation	0.83
13	12	20	3	Suburban	Creative Arts	0.39	0.16	0.18	0.32	0.53	0.36	0.71	Acts of Service	0.73
14	13	39	4	Urban	Creative Arts	0.81	0.61	0.59	0.39	0.35	0.69	0.76	Receiving Gifts	0.23
15	14	19	4	Urban	Engineering	0.65	0.46	0.41	0.15	0.85	0.42	0.73	Quality Time	0.75
16	15	41	3	Urban	Education	0.22	0.6	0.2	0.26	0.64	0.6	0.83	Receiving Gifts	0.71
17	16	47	3	Suburban	Law	0.77	0.75	0.15	0.63	0.68	0.62	0.33	Receiving Gifts	0.04
18	17	55	3	Suburban	Entrepreneurship	0.17	0.24	0.6	0.24	0.3	0.89	0.81	Words of Affirmation	0.68
19	18	19	2	Urban	Healthcare	0.28	0.37	0.89	0.22	0.42	0.53	0.43	Acts of Service	0.25
20	19	38	3	Urban	Engineering	0.42	0.69	0.54	0.43	0.41	0.32	0.6	Words of Affirmation	0.65
21	20	50	3	Urban	Law	0.52	0.48	0.73	0.19	0.59	0.93	0.82	Words of Affirmation	0.4
22	21	29	3	Urban	Entrepreneurship	0.69	0.73	0.25	0.86	0.2	0.77	0.56	Quality Time	0.47
23	22	39	2	Urban	Healthcare	0.29	0.5	0.83	0.67	0.25	0.41	0.55	Receiving Gifts	0.81
24	23	42	3	Urban	Marketing	0.55	0.31	0.54	0.21	0.36	0.67	0.94	Physical Touch	0.39
25	24	44	2	Rural	Science	0.3	0.32	0.19	0.5	0.63	0.22	0.6	Receiving Gifts	0.62
26	25	45	4	Urban	Education	0.59	0.32	0.54	0.42	0.25	0.78	0.62	Acts of Service	0.52

Figure 2.1: Extracto de los datos del Individuo A correspondientes a los 24 primeros emparejamientos evaluados. Se aprecian las variables demográficas y las métricas del modelo *Big Five*.

	A	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA
1	pair_id	b_age	b_education	b_location	b_career_field	b_career_ambition	b_openness	b_extraversion	b_agreeableness	b_conscientiousness	b_chronotype	b_spontaneity	b_love_language	b_emotional_expressiveness
2	1	20	3	Urban	Law	0.77	0.35	0.61	0.57	0.5	0.2	0.19	Quality Time	0.73
3	2	38	1	Urban	Law	0.44	0.71	0.31	0.2	0.57	0.45	0.56	Physical Touch	0.84
4	3	46	2	Rural	Finance	0.61	0.28	0.3	0.49	0.43	0.84	0.74	Physical Touch	0.48
5	4	20	2	Urban	Science	0.39	0.47	0.35	0.46	0.21	0.8	0.35	Receiving Gifts	0.41
6	5	22	3	Urban	Creative Arts	0.63	0.44	0.66	0.45	0.43	0.86	0.36	Acts of Service	0.5
7	6	41	2	Suburban	Education	0.68	0.56	0.12	0.36	0.04	0.14	1.0	Receiving Gifts	0.83
8	7	26	3	Rural	Science	0.58	0.61	0.26	0.26	0.49	0.36	0.43	Acts of Service	0.43
9	8	21	3	Urban	Marketing	0.62	0.58	0.73	0.4	0.42	0.62	0.71	Acts of Service	0.75
10	9	38	3	Rural	Finance	0.8	0.65	0.58	0.32	0.5	0.51	0.6	Words of Affirmation	0.86
11	10	36	4	Urban	Law	0.42	0.56	0.61	0.43	0.63	0.37	0.95	Physical Touch	0.32
12	11	34	4	Suburban	Education	0.29	0.2	0.1	0.2	0.68	0.45	0.44	Quality Time	0.69
13	12	55	3	Urban	Finance	0.57	0.31	0.14	0.13	0.68	0.22	0.53	Words of Affirmation	0.8
14	13	33	2	Urban	Healthcare	0.4	0.68	0.39	0.76	0.52	0.92	0.13	Acts of Service	0.19
15	14	51	1	Suburban	Marketing	0.26	0.33	0.7	0.55	0.25	0.66	0.14	Receiving Gifts	0.6
16	15	24	5	Suburban	Tech	0.69	0.88	0.15	0.09	0.23	0.66	0.17	Quality Time	0.62
17	16	35	2	Suburban	Science	0.48	0.89	0.22	0.97	0.07	0.62	0.49	Receiving Gifts	0.42
18	17	26	4	Urban	Marketing	0.72	0.56	0.62	0.06	0.34	0.35	0.74	Acts of Service	0.78
19	18	33	2	Urban	Law	0.26	0.72	0.27	0.53	0.25	0.29	0.42	Quality Time	0.31
20	19	25	3	Suburban	Healthcare	0.21	0.24	0.22	0.78	0.36	0.75	0.52	Words of Affirmation	0.85
21	20	52	3	Suburban	Creative Arts	0.46	0.37	0.52	0.23	0.07	0.69	0.93	Physical Touch	0.41
22	21	32	3	Suburban	Tech	0.18	0.34	0.28	0.67	0.36	0.46	0.19	Acts of Service	0.15
23	22	47	4	Urban	Law	0.5	0.66	0.6	0.34	0.67	0.75	0.25	Physical Touch	0.25
24	23	45	4	Suburban	Tech	0.57	0.32	0.56	0.27	0.8	0.76	0.06	Acts of Service	0.37
25	24	34	3	Urban	Marketing	0.26	0.58	0.64	0.26	0.69	0.62	0.6	Acts of Service	0.35

Figure 2.2: Extracto de los datos del Individuo B correspondientes a los 24 primeros emparejamientos. Nótese la obligatoria correspondencia del campo pair_id con el Individuo A para posibilitar el emparejamiento.

Más allá de las características predictoras de cada usuario individual, el sistema evalúa el resultado de la diada mediante sus variables objetivo. Siguiendo con la muestra de los 24 primeros registros analizados, la Figura 2.3 expone visualmente los campos `compatibility_score`, `compatible` y `relationship_longevity_months` para estas interacciones. En su conjunto, estas tres métricas son las encargadas de definir empíricamente qué tan “satisfactoria” ha resultado la relación, constituyendo la verdad terreno sobre la cual el algoritmo optimizará sus predicciones.

	A	AB	AC	AD
	pair_id	compatibility_score	compatible	relationship_longevity_months
1	1	43.5	0	60
2	2	60.4	0	59
3	3	74.3	1	84
4	4	58.0	0	70
5	5	69.8	1	68
6	6	67.2	1	64
7	7	66.0	1	119
8	8	43.8	0	60
9	9	72.9	1	69
10	10	60.9	0	85
11	11	63.4	0	65
12	12	54.1	0	53
13	13	57.6	0	57
14	14	57.8	0	70
15	15	46.7	0	70
16	16	65.3	1	80
17	17	65.9	1	72
18	18	96.6	1	113
19	19	67.2	1	59
20	20	69.3	1	103
21	21	45.6	0	71
22	22	75.2	1	98
23	23	69.6	1	72
24	24	53.0	0	34

Figure 2.3: Representación de las variables objetivo que definen el nivel de satisfacción y viabilidad de la relación, correspondientes a los 24 primeros registros del conjunto de datos.

2.4 Análisis de los datos y Detección de Sesgos

El Análisis Exploratorio de Datos (EDA) se ha enfocado en modelar las dinámicas de interacción intra-pareja, identificar patrones predictivos viables y detectar posibles sesgos (estadísticos o estructurales) en el conjunto de datos. Para ello se ejecutó un estudio sistemático apoyado en herramientas de visualización de datos avanzadas.

2.4.1 Análisis de Sesgos: Variables Objetivo (Score, Clases y Longevidad)

En esta primera serie de visualizaciones evaluamos el comportamiento y las distribuciones de nuestras tres variables objetivo (*targets*) principales (ver Figura 2.4):

- **Densidad del Score de Compatibilidad:** El primer gráfico revela que la puntuación base se ajusta a una distribución aproximadamente normal. No observamos polarizaciones extremas (sesgos hacia los extremos 0 o 100), lo que nos indica una distribución sana y equilibrada para utilizarla como métrica de regresión.
- **Balance de Clases (Éxito Relacional):** El gráfico central muestra un ligero desbalance natural en la etiqueta binaria *compatible*, con casi un 43% de casos de éxito frente a un 57% de fracasos. Al ser un desbalance moderado y altamente representativo de las dinámicas relacionales del mundo real, podemos prescindir de aplicar técnicas artificiales de sobremuestreo (como SMOTE) para nuestros modelos iniciales.

- **Distribución de Longevidad:** El histograma de la derecha revela cómo se distribuye el tiempo de vida de las relaciones en meses. Se observa una alta concentración de frecuencias en los primeros meses (rupturas tempranas típicas), disminuyendo gradualmente conforme avanza el tiempo. Nuestro modelo de regresión tendrá que aprender a interpretar esta asimetría temporal natural.

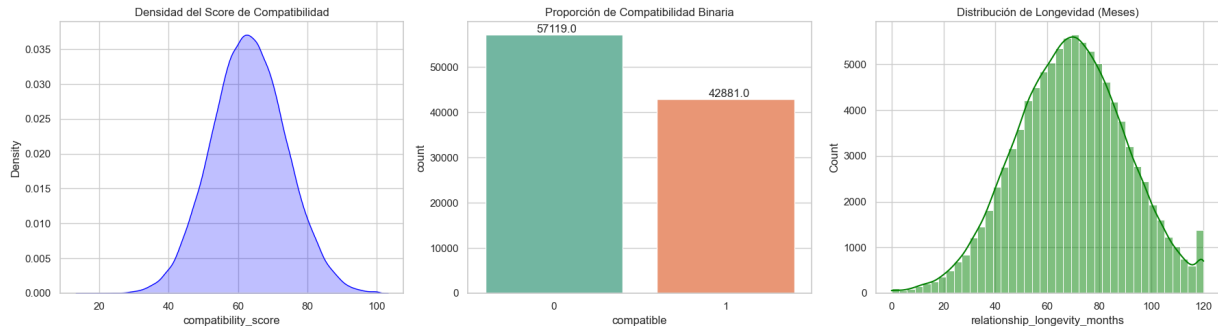


Figure 2.4: Distribución de las variables objetivo: densidad del score (izq.), balance de clases (centro) y longevidad (der.).

Para ahondar en esta distribución temporal, se procedió a agrupar los registros de longevidad en bloques de 5 meses. La Figura 2.5 ilustra claramente el **Sesgo de Supervivencia** inherente al problema. Existe una concentración masiva de rupturas (incompatibilidades, en rojo) en las primeras etapas. Esta visualización confirma que la predicción de longevidad será un reto algorítmico donde el modelo deberá aprender a discriminar qué factores otorgan resiliencia a largo plazo frente a los fracasos tempranos.

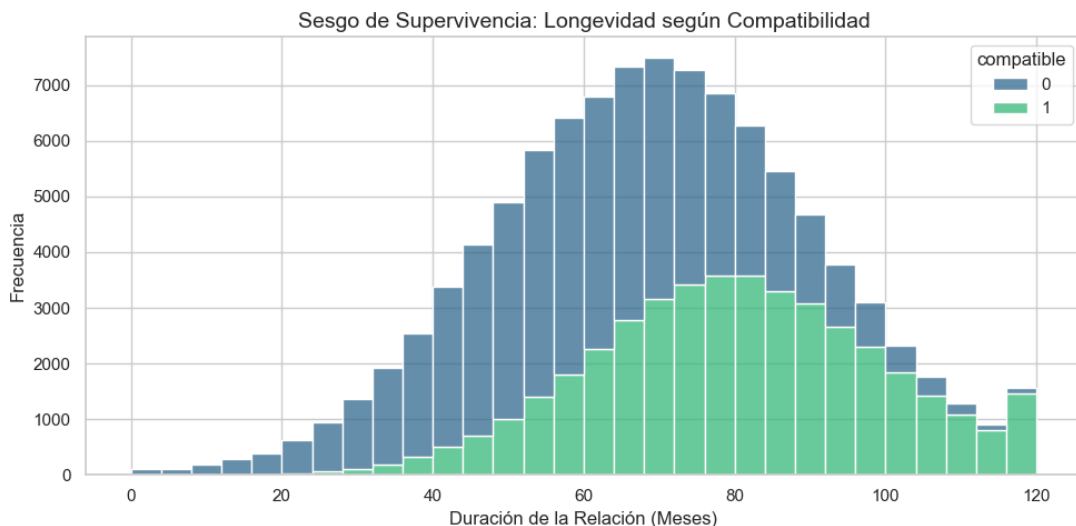


Figure 2.5: Distribución de registros según su longevidad agrupada en intervalos de 5 meses. Se aprecia la fuerte caída temprana de las relaciones incompatibles.

2.4.2 Análisis de Sesgos Estructurales: Ambición y Brecha de Edad

Estas visualizaciones exponen cómo las diferencias relativas o "brechas" absolutas afectan al éxito de la pareja, revelando patrones estructurales clave (ver Figura 2.6):

- **Afinidad en Ambición:** El gráfico de densidad KDE confirma que las parejas compatibles (representadas en azul) tienden a agruparse fuertemente cerca de la línea de identidad diagonal, lo que significa que poseen niveles de ambición profesional equiparables. Por el contrario, las parejas incompatibles (en rojo) presentan una mayor dispersión.
- **Diferencia de Edad (*Age Gap*):** El diagrama de violín nos muestra la distribución de la brecha de edad entre parejas exitosas y fallidas. Podemos observar cómo la forma de las distribuciones varía ligeramente, revelando que el algoritmo deberá aprender a sopesar esta diferencia absoluta como un posible factor de penalización relacional.

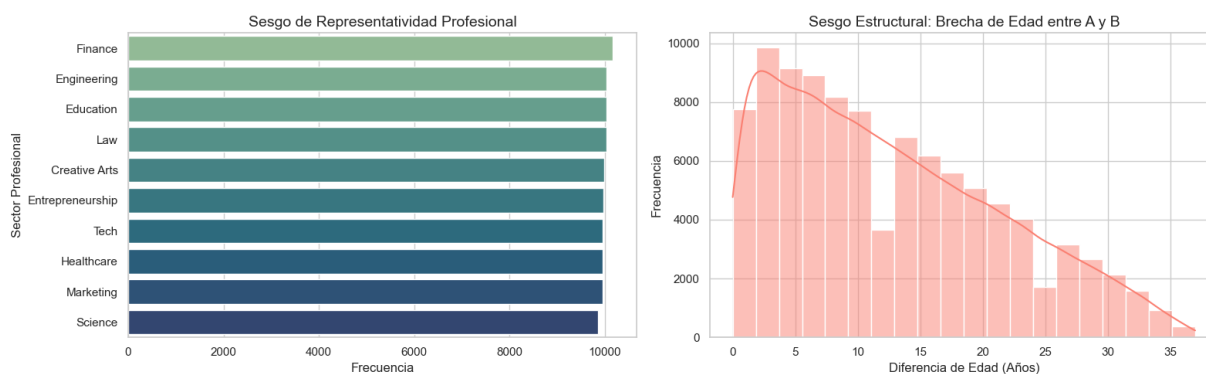


Figure 2.6: Análisis de sesgos estructurales: Afinidad en ambición profesional (izquierda) y diagrama de violín para la diferencia de edad (derecha).

2.4.3 Análisis de Sesgos de Interacción: Lenguajes del Amor

Esta visualización es crítica para entender cómo interactúan las variables categóricas entre ambos individuos. La Figura 2.7 muestra la matriz de contingencia térmica cruzando los lenguajes del amor.

- **Tasa de Compatibilidad por Cruce:** Este mapa de calor (*heatmap*) demuestra visualmente el principio sociológico de *homofilia*. Las celdas que conforman la diagonal principal (donde ambos individuos comparten exactamente el mismo lenguaje del amor) presentan tasas de éxito porcentual significativamente más altas. Esto nos indica de forma clara que el algoritmo deberá nutrirse de variables de interacción construidas mediante *Feature Engineering* para captar estas coincidencias.

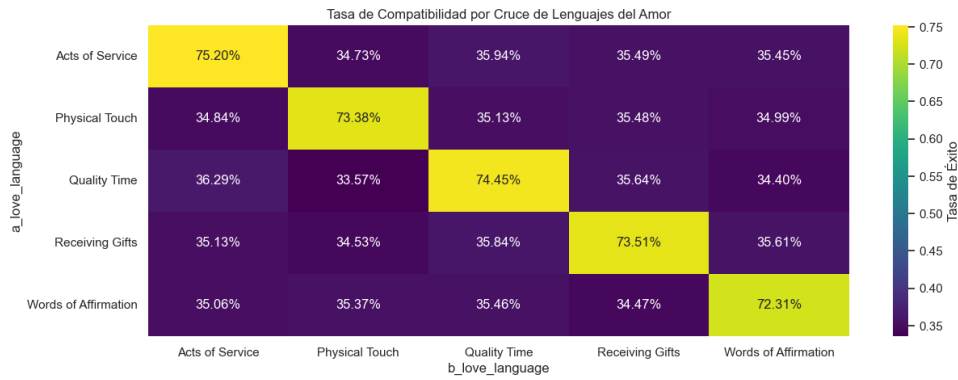


Figure 2.7: Matriz de contingencia térmica (*Heatmap*): Tasa de compatibilidad porcentual según el cruce de los lenguajes del amor entre ambos individuos.

2.4.4 Ingeniería de Características (*Feature Engineering*)

Con el fin de trasladar estas relaciones geométricas, mitigar sesgos de representación y proporcionar un marco matemático viable para los algoritmos de *Machine Learning*, se ejecutó un proceso profundo de Ingeniería de Características:

1. **Distancia Euclidiana de Personalidad:** Se procedió a calcular la norma euclidiana vectorial entre el conjunto de rasgos psicológicos (*Big Five* y expresividad emocional) de ambos individuos. Esta transformación matemática consolida múltiples variables en un predictor continuo robusto que penaliza las divergencias extremas de carácter.
2. **Disimilitud Continua:** Se derivaron nuevas métricas calculando las diferencias absolutas (brechas) en atributos numéricos clave, tales como la edad, la ambición profesional y el cronotipo.
3. **Interacciones Categóricas (Efecto Espejo):** Las coincidencias en variables nominales fueron codificadas como indicadores booleanos. El estudio determinó que presentar coincidencias exactas en estas variables incrementa notablemente la probabilidad de éxito. Por ejemplo, compartir el mismo campo profesional (`match_career = 1`) eleva la tasa de compatibilidad promedio hasta el 53.7%.

Las características derivadas se han consolidado en la Tabla 2.2, clasificadas en función de si su impacto resulta positivo o negativo para el algoritmo predictivo.

Característica (Feature)	Impacto	Descripción y Justificación Técnica
match_career	Positivo (+)	Coincidencia exacta en el sector profesional. Correlaciona positivamente con el éxito relacional.
match_love_language	Positivo (+)	Compartir el lenguaje del amor favorece la complementariedad y entendimiento afectivo.
personality_distance	Negativo (-)	Distancia euclidiana en el <i>Big Five</i> . A mayor distancia geométrica, disminuye la probabilidad de éxito.
diff_career_ambition	Negativo (-)	Brecha absoluta en ambición profesional. Las asimetrías severas actúan como factor fuertemente disruptivo.

Table 2.2: Diccionario de variables interactivas: características con pesos positivos y negativos

Para asegurar una integración óptima en la futura arquitectura de software y prevenir la transferencia indebida de información (*Data Leakage*), estas transformaciones se orquestaron a través de la clase `ColumnTransformer` (perteneciente a la librería *scikit-learn*). Este *pipeline* aplica una estandarización a las variables continuas y una codificación *One-Hot* a las categóricas, dejando el conjunto preparado para la etapa de inferencia y modelado.

2.5 Evaluación del rendimiento y Métricas

En el marco de la integración del algoritmo *Cupid* dentro de una arquitectura software real, hemos definido tres métricas operativas principales para evaluar el rendimiento del sistema, diseñadas específicamente para reflejar la experiencia relacional de los usuarios. Las métricas establecidas son:

- Dada una pareja, predecir si son compatibles o no, utilizando la métrica de compatibilidad.
- Dada una pareja, predecir si son compatibles y estimar su tiempo de duración (longevidad).
- Dada una persona, predecir qué otra persona es la más compatible con ella.

A continuación, se formaliza matemáticamente cada uno de estos enfoques:

2.5.1 1. Predicción de Compatibilidad Binaria

Para determinar si dos individuos son compatibles, el sistema genera primero una métrica de compatibilidad continua. Sea u_A y u_B los vectores de características de la Persona A y la Persona B. Definimos la función de compatibilidad del modelo como $S(u_A, u_B) \in [0, 100]$.

La predicción binaria final $\hat{y}$ (compatible o no compatible) se formaliza aplicando un umbral de decisión τ (típicamente establecido en 65, según la distribución empírica del dataset):

$$\hat{y}_{A,B} = \begin{cases} 1 & \text{si } S(u_A, u_B) \geq \tau \\ 0 & \text{si } S(u_A, u_B) < \tau \end{cases} \quad (2.1)$$

El rendimiento de esta predicción binaria sobre el conjunto de validación se evalúa mediante la Exactitud (*Accuracy*) y el *F1-Score*, contrastando $\hat{y}_{A,B}$ con el éxito real de la pareja, donde el *score* continuo S actúa como el motor discriminador.

2.5.2 2. Estimación de Longevidad Relacional

De forma complementaria a la clasificación de la compatibilidad, el sistema debe ser capaz de estimar la duración temporal de la relación. Definimos la función de estimación de longevidad como $L(u_A, u_B)$, la cual proyecta el tiempo en meses.

La evaluación conjunta asume que la longevidad es un problema de regresión. Para cuantificar la precisión matemática de nuestra estimación respecto a la longevidad real $Y_{A,B}$, se define la función de pérdida mediante el Error Cuadrático Medio (RMSE), penalizando de forma severa las discrepancias extremas en la predicción del tiempo de duración:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N \left(Y_{A_i, B_i} - L(u_{A_i}, u_{B_i}) \right)^2} \quad (2.2)$$

Donde N representa el total de parejas evaluadas. Esta métrica permite comprender el error de la estimación en la misma escala temporal (meses) de la variable objetivo.

2.5.3 3. Recomendación (Matching óptimo)

El objetivo de mayor impacto en el producto no evalúa parejas de forma aislada. En su lugar, dada una persona específica u_A y un conjunto de candidatos disponibles $C = \{c_1, c_2, \dots, c_m\}$, el algoritmo debe **predecir qué persona es la más compatible**, presentando un *ranking* ordenado en orden descendente según $S(u_A, c_j)$.

Para evaluar la calidad de este emparejamiento múltiple, se adoptan métricas del estado del arte de los Sistemas de Recomendación:

- **Mean Reciprocal Rank (MRR):** Cuantifica la rapidez del sistema para ofrecer el emparejamiento óptimo en la interfaz, localizando la posición (o rango) de la *primera* persona verdaderamente compatible recomendada a u_A .

$$MRR = \frac{1}{|U|} \sum_{i=1}^{|U|} \frac{1}{rank_i} \quad (2.3)$$

Donde $|U|$ es el número total de usuarios consultados, y $rank_i$ representa la posición en la lista del

primer candidato compatible. Su maximización asegura que el usuario no tendrá que hacer *scroll* para encontrar a su mejor prospecto.

- **Mean Average Precision (MAP):** Complementando al MRR, el MAP evalúa la calidad global del ranking, calculando la Precisión Media (*AP*) para el top K de personas más compatibles recomendadas:

$$MAP = \frac{1}{|U|} \sum_{u=1}^{|U|} \left(\frac{1}{m_u} \sum_{k=1}^K P_u(k) \cdot rel_u(k) \right) \quad (2.4)$$

Donde m_u es el número de posibles parejas exitosas para u ; K es el límite de recomendaciones visualizadas; $P_u(k)$ es la precisión hasta la posición k ; y $rel_u(k)$ es 1 si el candidato en la posición k resulta compatible, y 0 en caso contrario.

El planteamiento conjunto de estas tres dimensiones garantiza que la optimización algorítmica responda con exactitud matemática a las necesidades funcionales del producto y la experiencia del usuario.

Modelado, Experimentación y Validación

3.1 Introducción

Este capítulo recoge la fase de modelado del proyecto: la selección, entrenamiento, ajuste y evaluación comparativa de modelos de *Machine Learning* aplicados al problema del *Cupid Algorithm*. Siguiendo la metodología de un artículo científico, se aborda en primer lugar el diseño experimental, para después presentar los resultados obtenidos y extraer conclusiones fundamentadas sobre qué modelo resulta más adecuado para cada tarea.

Como se definió en el capítulo anterior, el problema se divide en tres tareas predictivas de complejidad creciente:

1. **Predicción de Compatibilidad Binaria:** Dada una pareja, determinar si son compatibles o no (*variable objetivo*: compatible).
2. **Estimación de Longevidad Relacional:** Dada una pareja compatible, estimar la duración de su relación en meses (*variable objetivo*: relationship_longevity_months).
3. **Recomendación de Emparejamiento Óptimo:** Dada una persona, identificar qué candidato del conjunto es el más compatible con ella (sistema de *ranking* con métricas MRR y MAP).

En este capítulo se abordan en profundidad las dos primeras tareas, por ser las que permiten una experimentación más controlada y cuyos resultados son directamente comparables mediante métricas estándar. La tercera tarea queda definida metodológicamente en el Capítulo 2 y se reserva para trabajo futuro.

3.2 Diseño Experimental

3.2.1 Partición de Datos

El conjunto de datos, compuesto por **100.000 observaciones**, se dividió en tres subconjuntos mutuamente excluyentes siguiendo la proporción acordada en la planificación del proyecto:

- **Entrenamiento (*train*):** 70% → 70.000 muestras. Utilizado para ajustar los parámetros de cada modelo.
- **Validación (*validation*):** 15% → 15.000 muestras. Utilizado para comparar modelos y seleccionar el mejor sin contaminar el conjunto de test.
- **Test:** 15% → 15.000 muestras. Utilizado únicamente para la evaluación final del modelo seleccionado, garantizando la imparcialidad de la estimación de rendimiento.

Esta estrategia de partición triple previene el sesgo de selección de modelos que se produciría si el conjunto de test se utilizara repetidamente durante el proceso de comparación. La semilla aleatoria se fijó en `random_state=42` para garantizar la reproducibilidad de todos los experimentos.

3.2.2 Selección de Características

A partir de las 23 características derivadas mediante Ingeniería de Características (detalladas en la Sección 2.4), se aplicó un proceso de selección formal basado en **Información Mutua** (*Mutual Information*), una medida no paramétrica que cuantifica la dependencia estadística entre cada predictor y la variable objetivo, sin asumir linealidad.

- Para la **clasificación**, se empleó `mutual_info_classif`, seleccionando las **20 características** con mayor relevancia para predecir `compatible`.
- Para la **regresión**, se empleó `mutual_info_regression`, seleccionando las **16 características** más informativas para predecir `relationship_longevity_months`.

Entre las características con mayor puntuación de información mutua para la tarea de clasificación destacan: `overall_match_score`, `same_love_language`, `ambition_diff`, `similar_ambition`, `emotional_expressiveness_diff` y `personality_distance_sum`. Estos resultados son coherentes con el análisis exploratorio del Capítulo 2, que ya señalaba la homofilia en el lenguaje del amor y la proximidad en ambición como factores determinantes.

3.2.3 Modelos Evaluados

Siguiendo el criterio de diversidad metodológica, se seleccionaron **cinco algoritmos** para cada tarea, abarcando desde modelos lineales e interpretables hasta conjuntos (*ensembles*) más complejos. La Tabla 3.1 y la Tabla 3.2 recogen la configuración de hiperparámetros utilizada.

Para abordar el problema de predicción de compatibilidad, se han seleccionado distintos modelos de aprendizaje automático con el objetivo de comparar cómo se comportan enfoques diferentes sobre el mismo conjunto de datos. Se parte de un modelo lineal como la regresión logística, para evaluar si las variables generadas durante la ingeniería de características son suficientes para explicar la compatibilidad. A partir

de ahí, se incluyen modelos más complejos como árboles de decisión, métodos ensemble (Random Forest y Gradient Boosting) y KNN, que permiten capturar relaciones no lineales, interacciones complejas y similitudes entre perfiles.

Los hiperparámetros se han fijado siguiendo valores habituales en la práctica, buscando configuraciones razonables sin excesiva complejidad. En árboles se han limitado parámetros como la profundidad para evitar sobreajuste, en KNN se ha utilizado un número reducido de vecinos y en regresión logística una configuración estándar. No se ha realizado una optimización exhaustiva, ya que el objetivo es comparar modelos de forma equilibrada. Como mejora futura, sería interesante ajustar los hiperparámetros de manera más sistemática, especialmente en modelos más sensibles como Decision Tree, Random Forest, Gradient Boosting y KNN.

Modelo	Abreviatura	Hiperparámetros principales
Regresión Logística	LR	$C=1.0$, <code>random_state=42</code> ; preprocesado con <code>StandardScaler</code>
Random Forest	RF	<code>n_estimators=400</code> , <code>max_depth=12</code> , <code>random_state=42</code>
Árbol de Decisión	DT	<code>max_depth=8</code> , <code>random_state=42</code>
Gradient Boosting	GB	<code>n_estimators=300</code> , <code>learning_rate=0.05</code> , <code>max_depth=3</code> , <code>random_state=42</code>
K-Vecinos más Cercanos	KNN	<code>StandardScaler</code> ; parámetros por defecto de <code>scikit-learn</code>

Table 3.1: Configuración de los modelos evaluados para la tarea de clasificación binaria (compatible).

Para la predicción de la longevidad de la relación se han utilizado las versiones de regresión de los modelos empleados en clasificación, manteniendo así coherencia y facilitando la comparación entre ambos enfoques. En concreto, se han incluido modelos como Ridge, árboles de decisión, Random Forest, Gradient Boosting y KNN en su versión de regresión, con el objetivo de analizar cómo se comportan estas mismas familias al predecir una variable continua como la duración de la relación.

Los hiperparámetros se han ajustado de forma similar a la tarea de clasificación, utilizando valores conservadores para evitar configuraciones demasiado complejas. No se ha realizado una optimización exhaustiva, ya que el objetivo principal es comparar modelos de manera equilibrada. Como mejora futura, sería interesante estudiar el impacto de un ajuste más detallado, especialmente en modelos como Random Forest, Gradient Boosting y KNN.

Modelo	Abreviatura	Hiperparámetros principales
Regresión Ridge	Ridge	<code>random_state=42</code> ; regularización L2 con α por defecto
Random Forest	RF	<code>n_estimators=200</code> , <code>max_depth=10</code> , <code>random_state=42</code>
Árbol de Decisión	DT	<code>max_depth=10</code> , <code>random_state=42</code>
Gradient Boosting	GB	<code>n_estimators=200</code> , <code>learning_rate=0.05</code> , <code>max_depth=3</code> , <code>random_state=42</code>
K-Vecinos más Cercanos	KNN	parámetros por defecto de <code>scikit-learn</code>

Table 3.2: Configuración de los modelos evaluados para la tarea de regresión (`relationship_longevity_months`).

Todos los modelos se implementaron mediante *pipelines* de `scikit-learn`, integrando el preprocesado (`StandardScaler` donde procede) y el estimador en un único objeto, lo que garantiza la ausencia de *data leakage* entre las particiones de datos.

3.2.4 Métricas de Evaluación

Clasificación

Para la comparación de los modelos de clasificación se utilizaron las siguientes métricas calculadas sobre el conjunto de validación:

- **Exactitud (*Accuracy*):** Proporción de predicciones correctas sobre el total. Es una métrica de referencia general, aunque puede resultar engañosa en presencia de desbalance de clases.
- **Precisión (*Precision*):** Fracción de predicciones positivas que son verdaderos positivos. Penaliza los falsos positivos.
- **Sensibilidad (*Recall*):** Fracción de positivos reales detectados por el modelo. Penaliza los falsos negativos.
- **F1-Score:** Media armónica entre Precisión y Sensibilidad. Es la métrica principal de comparación al equilibrar ambas perspectivas, especialmente relevante dado el ligero desbalance de clases (43% compatible, 57% no compatible).
- **ROC-AUC:** Área bajo la curva ROC. Mide la capacidad discriminativa del modelo con independencia del umbral de decisión.

Regresión

Para la comparación de los modelos de regresión se emplearon:

- **Error Absoluto Medio (MAE):** Media de las diferencias absolutas entre predicción y valor real, expresado en meses. Es robusto frente a *outliers* y de fácil interpretación.

- **Raíz del Error Cuadrático Medio (RMSE):** Penaliza más severamente los errores grandes que el MAE. También se expresa en meses.
- **Coefficiente de Determinación (R^2):** Proporción de la varianza de la variable objetivo explicada por el modelo. Un valor de 1 indica predicción perfecta; 0 indica que el modelo no supera la predicción por la media.

3.3 Resultados de Clasificación: Predicción de Compatibilidad

La Tabla 3.3 presenta las métricas obtenidas por los cinco modelos evaluados sobre el conjunto de validación.

Modelo	Accuracy	Precisión	Recall	F1-Score	ROC-AUC
Regresión Logística	0,7053	0,6440	0,6993	0,6705	0,7838
Random Forest	0,7107	0,6575	0,6788	0,6680	0,7807
Árbol de Decisión	0,6902	0,6254	0,6920	0,6570	0,7614
Gradient Boosting	0,7151	0,6898	0,6096	0,6472	0,7841
KNN	0,7015	0,6662	0,6093	0,6365	0,7597

Table 3.3: Métricas de clasificación sobre el conjunto de validación (15.000 muestras). La métrica principal de selección de modelo es el F1-Score.

3.3.1 Análisis de Resultados de Clasificación

El análisis de la Tabla 3.3 revela varios patrones de interés:

- **Regresión Logística** logra el mayor F1-Score (0,6705) a pesar de ser el modelo más simple, lo que indica que el proceso de Ingeniería de Características ha transformado el espacio de características de forma que la frontera de decisión óptima es aproximadamente lineal.
- **Gradient Boosting** obtiene la mayor exactitud (0,7151) y el ROC-AUC más alto (0,7841), pero su F1-Score es el cuarto peor. Esto se explica porque sacrifica *recall* (0,6096) en favor de una mayor precisión (0,6898), comportamiento subóptimo para nuestro caso de uso, donde identificar todas las parejas compatibles es tan importante como evitar los falsos positivos.
- **Random Forest** ofrece el segundo mejor F1-Score (0,6680) y un rendimiento global consistente, siendo un modelo robusto aunque menos interpretable.
- **Árbol de Decisión** y **KNN** quedan por detrás en todas las métricas principales, siendo los candidatos descartados.

Las matrices de confusión obtenidas en los conjuntos de validación y test muestran un comportamiento consistente del modelo, lo que indica una buena capacidad de generalización.

Se observa que el modelo es capaz de clasificar correctamente un número elevado de instancias en ambas clases, manteniendo un equilibrio razonable entre verdaderos positivos y verdaderos negativos. No obstante, se aprecia una mayor presencia de falsos positivos frente a falsos negativos, lo que sugiere que el modelo tiende a predecir la compatibilidad en mayor medida de lo debido. Este comportamiento puede interpretarse como una cierta inclinación del modelo hacia decisiones más permisivas en términos de compatibilidad.

En conjunto, los resultados reflejan un rendimiento sólido y coherente con las métricas obtenidas previamente, validando la elección del modelo de regresión logística como solución adecuada para este problema.

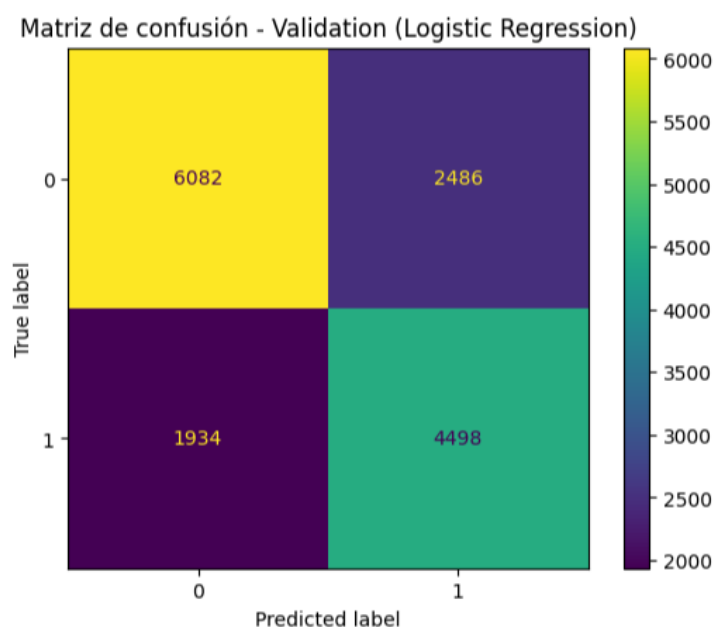


Figure 3.1: Matriz de confusión en validación

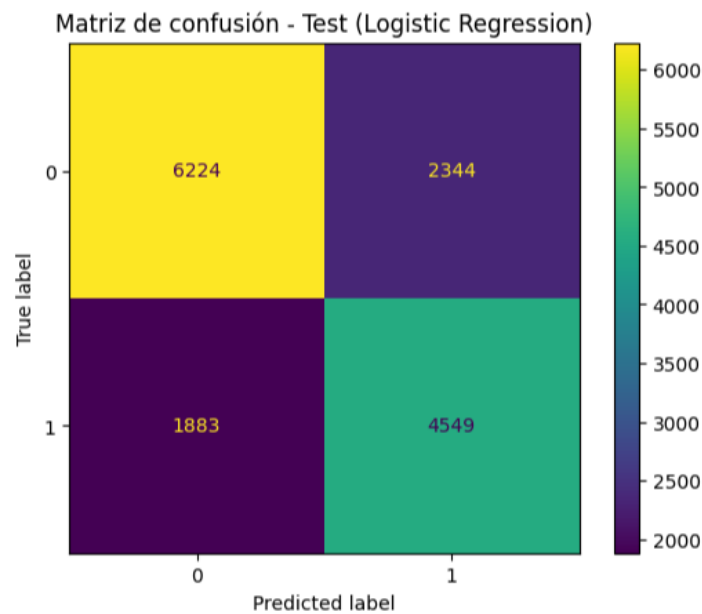


Figure 3.2: Matriz de confusión en test

3.3.2 Importancia relativa de variables por modelo en clasificación

La Figura 3.3 muestra la importancia relativa de las variables en los distintos modelos de clasificación. Se observa que `same_love_language` es la variable más relevante en todos los modelos, lo que indica que compartir el mismo lenguaje afectivo tiene un fuerte impacto en la compatibilidad.

También destacan variables relacionadas con diferencias entre individuos, como `emotional_expressiveness_diff`, `ambition_diff` y `spontaneity_diff`, especialmente en modelos no lineales. Esto sugiere que la compatibilidad depende en gran medida de la interacción entre perfiles.

En general, la importancia está distribuida entre varias variables, lo que refleja que se trata de un problema multifactorial.

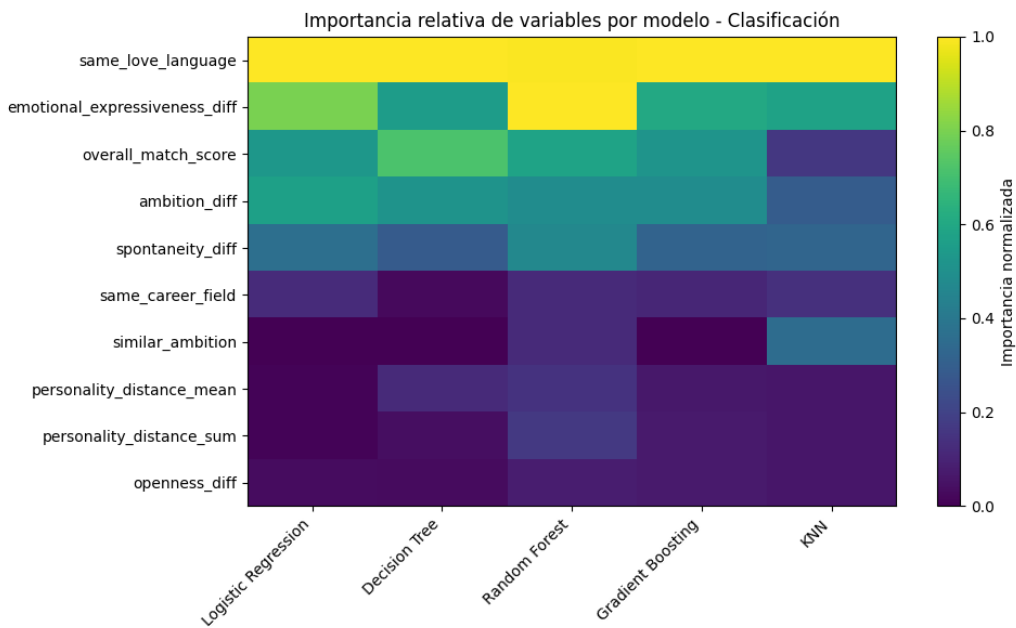


Figure 3.3: Importancia relativa de las variables en los modelos de clasificación. Los valores están normalizados por modelo y permiten comparar la relevancia de cada característica en la predicción de la compatibilidad.

3.3.3 Evaluación Final del Modelo Seleccionado: Regresión Logística

Tras seleccionar la Regresión Logística como mejor modelo basándonos en el F1-Score de validación, se procedió a su evaluación definitiva sobre el **conjunto de test**, que no había sido consultado en ningún momento del proceso de comparación. Los resultados, recogidos en la Tabla 3.4, confirman que el rendimiento se mantiene estable y no existe sobreajuste al conjunto de validación.

Conjunto	Accuracy	Precisión	Recall	F1-Score	ROC-AUC
Validación	0,7053	0,6440	0,6993	0,6705	0,7838
Test	0,7182	0,6599	0,7072	0,6828	0,7917

Table 3.4: Comparación del rendimiento de la Regresión Logística en validación y test. La mejora marginal en test indica ausencia de sobreajuste.

El modelo alcanza un **F1-Score de 0,6828** y un **ROC-AUC de 0,7917** en el conjunto de test, lo que supone una capacidad discriminativa sólida para un problema de compatibilidad relacional con variables de naturaleza continua y alta interacción entre ellas.

3.3.4 Interpretabilidad: Coeficientes del Modelo

Una de las ventajas fundamentales de la Regresión Logística frente a los modelos de *ensemble* es su interpretabilidad directa a través de los coeficientes. El análisis de los pesos asignados por el modelo confirma la coherencia con la teoría psicológica subyacente:

- La característica `emotional_expressiveness_diff` presenta el coeficiente negativo más significativo ($-0,5635$), indicando que las grandes diferencias en expresividad emocional son el factor más penalizador para la compatibilidad. Este hallazgo es consistente con la literatura sobre comunicación en parejas.
- Variables como `same_love_language`, `overall_match_score` y `high_personality_similarity` contribuyen positivamente, reforzando el principio de homofilia observado en el EDA.

3.4 Resultados de Regresión: Estimación de Longevidad

La Tabla 3.5 presenta las métricas obtenidas por los cinco modelos de regresión sobre el conjunto de validación.

Modelo	MAE (meses)	RMSE (meses)	R^2
Regresión Ridge	16,438	20,482	0,0804
Gradient Boosting	16,449	20,496	0,0792
Random Forest	16,510	20,575	0,0721
Árbol de Decisión	16,929	21,135	0,0209
KNN	18,099	22,627	$-0,1223$

Table 3.5: Métricas de regresión sobre el conjunto de validación. La métrica principal de selección es el MAE.

3.4.1 Análisis de Resultados de Regresión

El análisis de los resultados de la Tabla 3.5 pone de manifiesto la mayor dificultad intrínseca de la tarea de regresión frente a la clasificación:

- **Regresión Ridge** obtiene el mejor MAE (16,438 meses) y RMSE (20,482 meses), así como el R^2 más alto (0,0804), y es seleccionada como modelo ganador.
- **Gradient Boosting** se sitúa en segunda posición con métricas prácticamente idénticas a Ridge (MAE de 16,449 meses, diferencia de tan solo 0,011 meses), lo que sugiere que ambos modelos han alcanzado un techo de rendimiento similar con las características disponibles.
- **KNN** ofrece el peor rendimiento con diferencia, presentando incluso un R^2 negativo ($-0,1223$), lo que indica que su predicción es peor que la predicción naive basada en la media. Este comportamiento es esperable dado que KNN es sensible a la dimensionalidad y a la escala, y la longevidad es un fenómeno temporalmente irregular.
- El R^2 **reducido** de todos los modelos (máximo de 0,0804) es el hallazgo más significativo del experimento de regresión. Indica que las variables disponibles en el *dataset* explican una fracción limitada

de la varianza en la duración de las relaciones, lo que es coherente con la naturaleza del problema: la longevidad relacional depende de factores dinámicos, contextuales y emocionales que difícilmente se capturan con un perfil estático de características.

3.4.2 Importancia relativa de variables por modelo en regresión

La Figura 3.4 muestra la importancia relativa de las variables en los distintos modelos de regresión. A diferencia del caso de clasificación, no se observa una variable dominante. La importancia se distribuye entre múltiples características.

Destacan variables como `overall_match_score`, `same_love_language` y diferencias en rasgos individuales como `emotional_expressiveness_diff` o `spontaneity_diff`, especialmente en modelos no lineales como Random Forest y Gradient Boosting.

Sin embargo, la variabilidad en la importancia entre modelos y la ausencia de patrones consistentes reflejan la dificultad del problema y la limitada capacidad explicativa de las variables disponibles.

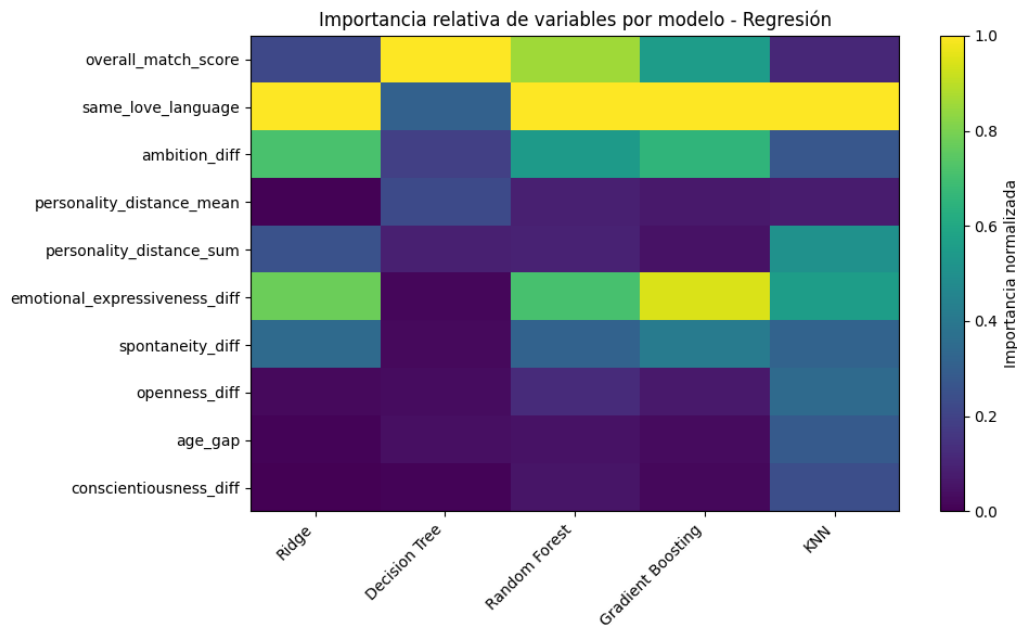


Figure 3.4: Importancia relativa de las variables en los modelos de regresión. Los valores están normalizados por modelo y muestran una distribución más homogénea de la relevancia, sin variables claramente dominantes.

3.4.3 Evaluación Final del Modelo Seleccionado: Regresión Ridge

Tras seleccionar Ridge como mejor modelo, su evaluación sobre el **conjunto de test** se presenta en la Tabla 3.6.

Conjunto	MAE (meses)	RMSE (meses)	R^2
Validación	16,438	20,482	0,0804
Test	16,271	20,334	0,0758

Table 3.6: Comparación del rendimiento de la Regresión Ridge en validación y test.

El modelo obtiene un **MAE de 16,271 meses** y un **RMSE de 20,334 meses** en test. Considerando que la variable objetivo oscila entre 0 y 120 meses, el error absoluto medio representa aproximadamente un 13,6% del rango total, lo cual, aunque modesto, refleja correctamente la dificultad del problema.

3.5 Discusión

3.5.1 Comparativa entre Clasificación y Regresión

Los resultados ponen de manifiesto una asimetría clara entre las dos tareas abordadas:

- La **predicción de compatibilidad binaria** (clasificación) alcanza métricas razonables con la Regresión Logística (F1=0,6828, ROC-AUC=0,7917). El proceso de *Feature Engineering* resulta especialmente efectivo para esta tarea, al transformar variables individuales en indicadores de similitud y divergencia que el modelo lineal puede explotar directamente.
- La **estimación de longevidad** (regresión) resulta significativamente más compleja, con un R^2 máximo de 0,0804 en validación. Esta limitación no es un fallo del modelado en sí, sino una evidencia de que los factores que determinan la duración de una relación trascienden los atributos de perfil estático. Eventos vitales, adaptabilidad, comunicación dinámica y contexto socioeconómico son variables ausentes del *dataset* pero determinantes en la realidad.

3.5.2 Elección de Modelos Simples frente a Modelos Complejos

Un resultado llamativo es que en ambas tareas los modelos lineales (Regresión Logística y Ridge) superan o igualan a los *ensembles* más complejos (Random Forest, Gradient Boosting). Este fenómeno se explica por:

1. **Calidad del Feature Engineering:** Las 20–23 características derivadas ya incorporan la no linealidad del problema (distancias, interacciones categóricas, indicadores compuestos), dejando al modelo una tarea de agregación lineal.
2. **Generalización:** Los modelos lineales generalizan mejor en espacios de alta dimensión cuando las señales son débiles y ruidosas, que es precisamente el caso de la longevidad relacional.
3. **Navaja de Ockham:** En igualdad de condiciones predictivas, el modelo más simple y explicable es preferible, especialmente en sistemas que deben ser auditados o integrados en entornos con requisitos de transparencia.

3.6 Conclusiones

El proceso de experimentación y validación ha permitido extraer las siguientes conclusiones:

1. **Modelo recomendado para compatibilidad:** La **Regresión Logística** es el modelo más adecuado para predecir si una pareja es compatible, logrando un F1-Score de 0,6828 y ROC-AUC de 0,7917 en test. Su interpretabilidad a través de los coeficientes proporciona además valor explicativo al sistema.
2. **Modelo recomendado para longevidad:** La **Regresión Ridge** es la mejor opción para estimar la duración de la relación, con un MAE de 16,271 meses en test. No obstante, el bajo R^2 advierte de que esta predicción debe emplearse con cautela y comunicarse al usuario con intervalos de incertidumbre.
3. **Importancia del Feature Engineering:** La ingeniería de características ha sido el factor más determinante del rendimiento. Los indicadores derivados (distancia de personalidad, coincidencia de lenguaje del amor, *overall match score*) concentran la mayor parte de la señal predictiva.
4. **Límites del dataset:** El R^2 reducido en regresión señala que para mejorar sustancialmente la predicción de longevidad sería necesario enriquecer el *dataset* con variables dinámicas (comportamiento en la plataforma, retroalimentación de los usuarios) o plantear el problema como supervivencia (*survival analysis*).
5. **Reproducibilidad:** Todos los experimentos se han ejecutado con `random_state=42` y los resultados son completamente reproducibles a partir del *notebook* adjunto al repositorio del proyecto.

Los resultados obtenidos ponen de manifiesto una diferencia clara entre ambos problemas: mientras que la compatibilidad puede modelarse de forma razonable a partir de variables estructuradas y relaciones entre perfiles, la longevidad presenta una mayor complejidad y menor capacidad explicativa con la información disponible.

Esto refuerza la idea de que no todos los problemas dentro de un mismo dominio presentan el mismo grado de dificultad, y destaca la importancia de analizar no solo el rendimiento de los modelos, sino también la naturaleza de la variable objetivo y la información disponible.

Como líneas futuras de trabajo, se plantea la posibilidad de incorporar nuevas variables, aplicar técnicas de optimización de hiperparámetros o utilizar modelos más complejos que permitan capturar relaciones no lineales más profundas, especialmente en la tarea de regresión.

Despliegue y Puesta en Producción

4.1 Introducción

Este capítulo describe el proceso de transición del sistema *CORE-MP* desde el entorno de experimentación —basado en *notebooks* de Jupyter— hasta un despliegue real y reproducible en producción. Esta fase, conocida en la industria como *MLOps* (*Machine Learning Operations*), constituye uno de los pasos más críticos en el ciclo de vida de cualquier modelo de aprendizaje automático: sin ella, los modelos entrenados permanecen confinados en el entorno del científico de datos y no generan valor real para el usuario final.

La transición a producción implica resolver una serie de retos técnicos que van más allá del modelado:

- **Reproducibilidad:** garantizar que el entorno de ejecución del modelo es idéntico en cualquier máquina, independientemente del sistema operativo o las dependencias instaladas localmente.
- **Separación de responsabilidades:** distinguir claramente entre el proceso de entrenamiento (que ocurre de forma periódica o puntual) y el servicio de inferencia (que debe estar disponible de forma continua).
- **Consistencia del preprocesado:** asegurar que las transformaciones aplicadas a los datos en tiempo de entrenamiento se replican de forma exacta en tiempo de inferencia, evitando el denominado *training-serving skew*.
- **Observabilidad:** instrumentar el sistema para medir su rendimiento en producción y detectar degradaciones del modelo a lo largo del tiempo.

La solución adoptada en este proyecto se basa en tres pilares tecnológicos: **FastAPI** como framework para la construcción de la API REST, **Docker** como mecanismo de containerización y **una arquitectura modular en Python** que separa limpiamente cada responsabilidad del sistema.

4.2 Arquitectura del Sistema

4.2.1 Visión General

El sistema de producción se compone de **dos contenedores Docker independientes** con ciclos de vida distintos:

- **Contenedor de entrenamiento (train):** ejecuta el pipeline completo de ingesta, ingeniería de características y entrenamiento de modelos. Se trata de un contenedor de *ejecución única*: arranca, entrena, serializa los modelos en disco y termina. No es un servicio persistente.
- **Contenedor de inferencia (api):** carga los modelos serializados desde el volumen compartido y expone una API REST que permanece activa indefinidamente, respondiendo peticiones de predicción en tiempo real.

La comunicación entre ambos contenedores se realiza exclusivamente a través de un **volumen Docker compartido** (`models_output/`), en el que el contenedor de entrenamiento escribe los artefactos del modelo y el contenedor de inferencia los lee. Este mecanismo de desacoplamiento garantiza que ningún contenedor necesita conocer los detalles internos del otro.

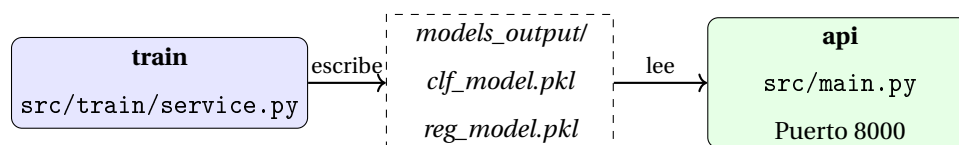


Figure 4.1: Arquitectura de dos contenedores del sistema CORE-MP. El volumen compartido actúa como contrato entre el pipeline de entrenamiento y el servicio de inferencia.

4.2.2 Justificación del Diseño

La decisión de separar entrenamiento e inferencia en contenedores independientes responde a principios fundamentales de arquitecturas de sistemas en producción:

1. **Independencia de ciclos de vida:** el reentrenamiento del modelo no requiere interrumpir el servicio de inferencia. Basta con ejecutar el contenedor de entrenamiento, sustituir los artefactos en el volumen y reiniciar la API.
2. **Menor superficie de ataque:** el contenedor de inferencia no incluye las dependencias de entrenamiento (e.g., `scikit-learn` completo con sus utilidades de selección de modelos), lo que reduce el tamaño de la imagen y el número de dependencias externas.
3. **Escalabilidad diferenciada:** en un entorno de producción real, el servicio de inferencia puede escalarse horizontalmente (múltiples réplicas) mientras el entrenamiento permanece como proceso *batch* único.

4.3 Pipeline de Ingeniería de Características en Producción

4.3.1 El Principio Stateless / Stateful

Una de las decisiones de diseño más críticas en la transición a producción es la gestión del preprocesado de datos. En el sistema implementado, todas las transformaciones de ingeniería de características se centralizan en el módulo `src/features/service.py` y se clasifican según su naturaleza:

- **Transformaciones *stateless* (sin estado):** operaciones que se calculan directamente a partir de los datos de entrada, sin requerir parámetros ajustados previamente. En este proyecto, la totalidad de las 23 características derivadas (diferencias absolutas entre perfiles, indicadores binarios de coincidencia, distancias de personalidad y el `overall_match_score`) son de este tipo. La función `create_pair_features()` encapsula estas transformaciones y se invoca de forma idéntica tanto durante el entrenamiento como durante la inferencia.
- **Transformaciones *stateful* (con estado):** operaciones que requieren parámetros calculados sobre el conjunto de entrenamiento y que deben serializarse para ser reutilizados en inferencia, como escaladores (`StandardScaler`) o límites de *outliers*. En este proyecto, el `StandardScaler` se integra dentro del *pipeline* de `scikit-learn` del modelo de clasificación y se serializa conjuntamente con él, garantizando que los parámetros de escalado aprendidos en entrenamiento se aplican de forma idéntica en inferencia.

Esta separación es la que previene el *training-serving skew*: el error más común y costoso en sistemas de *Machine Learning* en producción, que se produce cuando las transformaciones aplicadas a los datos en tiempo de inferencia difieren —aunque sea sutilmente— de las aplicadas durante el entrenamiento.

4.4 Implementación de la API REST

4.4.1 Framework y Tecnología

La API de inferencia se implementa con **FastAPI** [`fastapi`], un framework moderno de Python para la construcción de APIs REST con alto rendimiento, validación automática de tipos mediante **Pydantic** y generación automática de documentación interactiva en formato OpenAPI (accesible en `/docs`).

La elección de FastAPI frente a alternativas como Flask o Django REST Framework se justifica por:

- **Validación declarativa:** los modelos Pydantic definen el contrato de entrada y salida de la API de forma explícita, rechazando automáticamente peticiones malformadas con mensajes de error descriptivos.
- **Rendimiento:** FastAPI está construido sobre `Starlette` y `uvicorn`, alcanzando rendimientos comparables a frameworks asíncronos de otros lenguajes.

- **Documentación automática:** la especificación OpenAPI se genera sin código adicional, facilitando la integración con sistemas externos.

4.4.2 Gestión del Ciclo de Vida

Un aspecto crítico de la API es la carga de los modelos. Se adopta el patrón *fail-fast*: los modelos se cargan **una única vez** en el arranque del servidor, mediante el gestor de ciclo de vida (*lifespan*) de FastAPI, y se almacenan en `app.state`. Si alguno de los ficheros `.pkl` no existe en el momento del arranque, el servidor falla inmediatamente con un error claro, evitando que la API quede activa sin modelos disponibles. Este comportamiento es preferible a detectar el fallo en el momento de la primera petición.

La inyección de dependencias (Depends) permite que los *endpoints* accedan al predictor y al *tracker* de métricas sin acoplarse a variables globales, mejorando la testabilidad y la claridad del código.

4.4.3 Endpoints Disponibles

La API expone tres *endpoints*, agrupados en dos dominios funcionales:

Método	Ruta	Descripción
POST	<code>/predict</code>	Recibe los perfiles de dos personas y devuelve la predicción de compatibilidad y longevidad estimada.
GET	<code>/health</code>	Comprueba que el servicio está activo y los modelos están cargados.
GET	<code>/metrics</code>	Devuelve métricas acumuladas de latencia: número de peticiones, latencia total y latencia media.

Table 4.1: Endpoints de la API REST del sistema CORE-MP.

4.4.4 Esquema de Entrada y Salida

El *endpoint* principal POST `/predict` acepta un objeto JSON con los perfiles de las dos personas (`person_a` y `person_b`), cada uno con los 13 atributos originales del *dataset*. La respuesta incluye:

- **compatible:** clasificación binaria (*true/false*), derivada del umbral de probabilidad $p \geq 0,5$.
- **compatibility_probability:** probabilidad estimada por la Regresión Logística, en el intervalo $[0, 1]$.
- **estimated_longevity_months:** duración estimada de la relación en meses, obtenida invirtiendo la transformación logarítmica aplicada durante el entrenamiento ($\hat{y} = e^{\hat{y}_{\log}} - 1$).
- **overall_match_score:** puntuación global de compatibilidad calculada durante el *feature engineering*, con valor en $[0, 1]$.
- **latency_ms:** tiempo de procesamiento de la petición en milisegundos.

4.5 Containerización con Docker

4.5.1 Imágenes Docker

Cada uno de los dos contenedores se construye a partir de una imagen Docker independiente, definida en su propio *Dockerfile*:

- `Dockerfile.train`: utiliza `python:3.12-slim` como imagen base. Instala las dependencias del fichero `requirements.txt` y ejecuta el módulo `src.train.service` al arrancar, lo que desencadena el pipeline completo de entrenamiento.
- `Dockerfile.api`: utiliza la misma imagen base. Arranca el servidor `uvicorn` apuntando al módulo `src.main:app`, exponiendo el puerto 8000.

En ambos casos, las dependencias se instalan como primera capa de la imagen, aprovechando el sistema de caché de Docker: si el fichero `requirements.txt` no cambia entre construcciones, esta capa se reutiliza, reduciendo el tiempo de *build* significativamente.

4.5.2 Orquestación con Docker Compose

El fichero `docker-compose.yml` orquesta ambos servicios y gestiona la comunicación a través de volúmenes:

- El servicio `train` monta el directorio `./data/raw` (solo lectura) para acceder al *dataset*, y el directorio `./models_output` (lectura y escritura) para depositar los modelos entrenados.
- El servicio `api` monta el directorio `./models_output` (solo lectura) para cargar los modelos, y expone el puerto `8000:8000` al *host*.
- Las rutas de los modelos y del *dataset* se inyectan a través de variables de entorno, definidas centralmente en `src/config.py`, lo que permite sobrescribir cualquier parámetro sin modificar el código fuente.

El flujo de ejecución recomendado es secuencial: primero se lanza el contenedor de entrenamiento hasta su terminación y, a continuación, se arranca el servicio de API:

1. `docker compose run -rm train`
2. `docker compose up api`

4.6 Estructura Modular del Código

4.6.1 Organización por Dominio Funcional

El código fuente del sistema sigue una arquitectura orientada a dominios funcionales, inspirada en las convenciones de proyectos FastAPI de nivel empresarial.

Esta estructura garantiza que cada componente tenga una única responsabilidad (*Single Responsibility Principle*) y que los cambios en un dominio no afecten a los demás. Por ejemplo, una modificación en el esquema de la respuesta de la API solo requiere editar `src/infer/schemas.py`, sin alterar la lógica de predicción ni la de monitoreo.

4.7 Estrategia de Monitoreo

4.7.1 Métricas de Latencia

El monitoreo de un sistema de *Machine Learning* en producción es una condición necesaria para detectar degradaciones del modelo (*model drift*) o problemas de rendimiento de la infraestructura. En el sistema implementado, cada petición al *endpoint* `/predict` registra su tiempo de procesamiento, acumulado en el objeto `MetricsTracker` almacenado en `app.state`.

La **latencia media** del sistema se calcula como:

$$\bar{L} = \frac{\sum_{i=1}^N (t_{\text{respuesta},i} - t_{\text{solicitud},i})}{N} \quad (4.1)$$

donde $t_{\text{solicitud},i}$ es el instante en que se recibe la i -ésima petición, $t_{\text{respuesta},i}$ es el instante en que se envía la respuesta correspondiente, y N es el número total de peticiones procesadas.

Esta métrica es accesible en tiempo real mediante `GET /metrics`, que devuelve el número total de peticiones, la latencia acumulada y la latencia media, todo ello calculado de forma **thread-safe** mediante un `threading.Lock`, garantizando la consistencia de los contadores bajo carga concurrente.

4.7.2 Comprobación de Estado (*Health Check*)

El *endpoint* `GET /health` devuelve el estado operativo del servicio, indicando si los modelos están cargados correctamente y la versión de la API. Este *endpoint* puede ser consumido por herramientas de orquestación (como Kubernetes o Docker Swarm) para determinar si el contenedor está listo para recibir tráfico (*readiness probe*).

4.8 Conclusiones

El proceso de despliegue del sistema *CORE-MP* ha permitido materializar los modelos entrenados en el Capítulo 3 en un servicio real, reproducible y observable. Las principales conclusiones son:

1. **La separación de entrenamiento e inferencia** en contenedores independientes es el principio más importante del despliegue. Permite actualizar el modelo sin interrumpir el servicio y escalar ambos componentes de forma independiente según sus necesidades.

2. El **patrón *stateless*** en el *feature engineering* garantiza la consistencia entre entrenamiento e inferencia. Al centralizar las transformaciones en `src/features/service.py` y aplicarlas de forma idéntica en ambos contextos, se elimina por diseño la posibilidad de *training-serving skew*.
3. La **arquitectura modular** organizada por dominio funcional (`features`, `train`, `infer`, `monitor`) mejora la mantenibilidad y permite que equipos distintos trabajen sobre componentes independientes sin interferencias.
4. El **patrón *fail-fast*** en la carga de modelos garantiza que el sistema no queda activo en un estado inconsistente: si los artefactos del modelo no están disponibles en el arranque, el servidor falla inmediatamente con un error claro y descriptivo.
5. El **monitoreo de latencia** incorporado desde el primer día, con métricas accesibles vía API, sienta las bases para una evolución hacia sistemas de detección de *drift* más sofisticados, que podrían alertar cuando la distribución de los datos de entrada en producción se aleja de la distribución de entrenamiento.

Como líneas de trabajo futuro en el ámbito del despliegue, se plantea la integración de **MLflow** para el registro y versionado de experimentos, el uso de **MinIO** como almacenamiento de artefactos compatible con S3, y la orquestación del sistema sobre **Kubernetes** para entornos de producción con alta disponibilidad y escalado automático.

Monitorización y Feedback Loop

5.1 Introducción

El despliegue de un modelo de *Machine Learning* en producción no es el final del proceso, sino el inicio de su mantenimiento continuo. Un modelo puede degradarse gradualmente cuando la distribución de los datos reales se aleja de la distribución de entrenamiento (fenómeno conocido como *data drift*) sin que ningún error de código lo evidencie. Este capítulo describe la capa de monitorización del sistema *CORE-MP*, compuesta por métricas operativas con **Prometheus** y **Grafana**, detección de drift con **Evidently**, un feedback loop automatizado con **Apache Airflow** y alertas vía **Telegram**. El conjunto forma un ciclo de *Continuous Training* (CT) autónomo que mantiene la calidad del modelo en producción sin intervención manual.

5.2 Arquitectura de Monitorización

La capa de monitorización amplía el sistema del despliegue y puesta en producción con cuatro contenedores adicionales, todos orquestados en `docker-compose.yml`:

Contenedor	Imagen	Puerto	Función
api	Dockerfile.api	8000	Inferencia + exportación de métricas
prometheus	prom/prometheus	9092	Recolección y almacenamiento de métricas
grafana	grafana/grafana	3000	Visualización y alertas
pushgateway	prom/pushgateway	9091	Métricas <i>batch</i> del DAG de Airflow
airflow	apache/airflow:2.9.0	8080	Orquestación del feedback loop
train	Dockerfile.train	—	Reentrenamiento (on-demand)

Table 5.1: Contenedores del stack de monitorización de CORE-MP.

La Figura 5.1 muestra el flujo de información: Prometheus *escrapea* la API cada 15 segundos; el DAG de Airflow empuja sus métricas de drift al Pushgateway, que Prometheus también recolecta; Grafana consume ambas fuentes en un *dashboard* unificado.

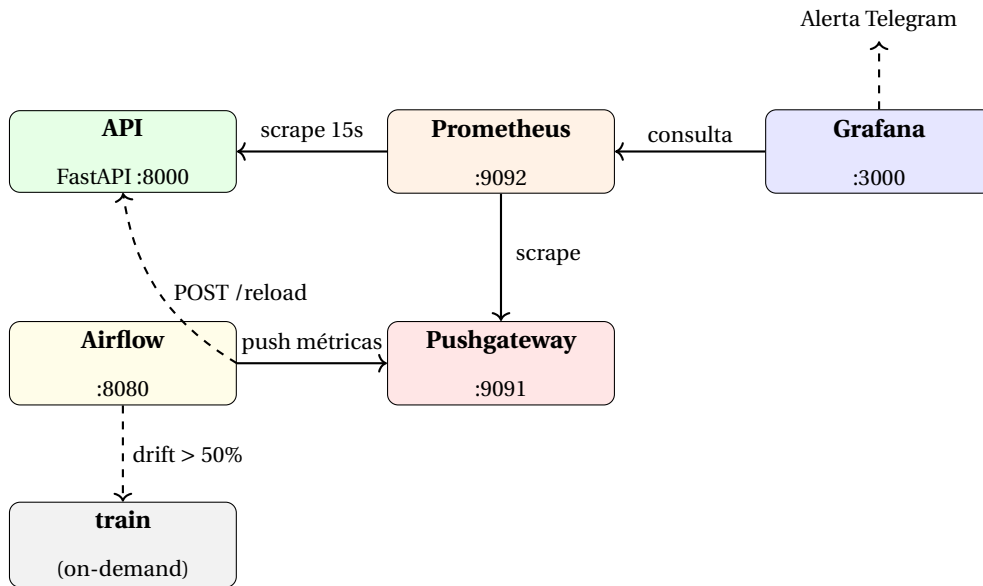


Figure 5.1: Flujo de datos del stack de monitorización. Flechas sólidas: métricas; discontinuas: acciones de control del feedback loop.

5.3 Métricas Operativas y Dashboard

La API instrumenta todas las rutas HTTP mediante `prometheus-fastapi-instrumentator`, que expone histogramas de latencia y contadores de peticiones en `GET /metrics/prometheus`. Adicionalmente, se define el contador personalizado `predictions_total`, etiquetado por resultado (`compatible=true/false`), y el módulo `src/monitor/service.py` mantiene un `MetricsTracker` *thread-safe* (con `threading.Lock`) que calcula la latencia media acumulada:

$$\bar{L} = \frac{\sum_{i=1}^N (t_{\text{respuesta},i} - t_{\text{solicitud},i})}{N} \quad (5.1)$$

Grafana se provisiona automáticamente desde `monitoring/grafana/provisioning/` con un *dashboard* de refresco de 10 segundos y cuatro paneles (Tabla 5.2). El umbral de P95 está fijado en 0,5 s: si se supera, el panel se colorea en rojo. Para garantizar la idempotencia del provisioning, el *datasource* de Prometheus tiene un UID fijo (`uid: prometheus`), de modo que los paneles encuentran su fuente de datos incluso tras recrear el contenedor de Grafana.

Panel	Tipo	Consulta PromQL
Request Rate (req/s)	Serie temporal	<code>sum(rate(http_request_duration_seconds_count[1m]))</code>
Latencia P95 (s)	Serie temporal	<code>histogram_quantile(0.95, sum by (le) (rate(...[5m])))</code>
Total predicciones	Estadístico	<code>predictions_total</code>
Tasa de errores 5xx	Serie temporal	<code>sum(rate(...{status=~"5.."}[1m]))</code>

Table 5.2: Paneles del dashboard de Grafana para el sistema CORE-MP.

La Figura 5.2 muestra el dashboard en funcionamiento con tráfico real: el panel *Request Rate* refleja las peticiones generadas durante la validación, *Latencia P95* se mantiene en torno a 95 ms y el contador de predicciones acumula 70 llamadas sin errores 5xx.

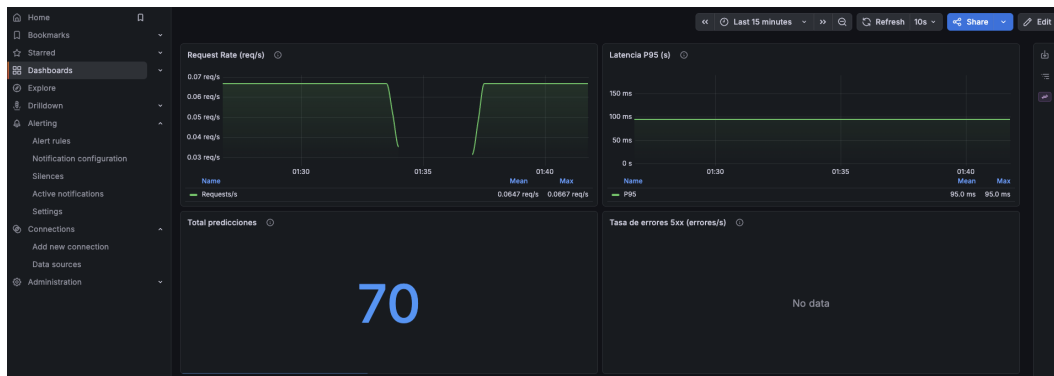


Figure 5.2: Dashboard de Grafana con los cuatro paneles de monitorización operativa de CORE-MP en tiempo real.

5.4 Detección de Drift de Datos

El *data drift* se produce cuando la distribución de los datos de entrada en producción se aleja de la distribución de entrenamiento, degradando métricas como F1 o MAE aunque el código no tenga errores. La detección se implementa sobre seis características derivadas seleccionadas en el modelado:

- `same_love_language`
- `ambition_diff`
- `emotional_expressiveness_diff`
- `spontaneity_diff`
- `overall_match_score`
- `personality_distance_sum`

Tras cada predicción, `src/monitor/drift_logger.py` calcula estas seis features y las añade incrementalmente a `current_traffic.csv`. Los datos de referencia (`reference_data.csv`) se generan una única vez antes de la puesta en marcha, tomando 2.000 registros del *dataset* original con las mismas fórmulas, para garantizar la comparabilidad.

La detección propiamente dicha la realiza **Evidently** mediante tests estadísticos (Kolmogorov-Smirnov para variables continuas, χ^2 para categóricas). El análisis requiere al menos 50 muestras (`MIN_SAMPLES = 50`) para evitar resultados espurios. Si la proporción de features con drift supera el **50%** (`DRIFT_THRESHOLD = 0.5`), se activa el reentrenamiento. En cualquier caso, se empujan dos métricas al Pushgateway (`core_mp_drift_detected` y `core_mp_drift_share`) para su visualización en Grafana.

5.5 Feedback Loop con Airflow y Alertas

El feedback loop se implementa como el DAG `core_mp_feedback_loop` en Apache Airflow, con ejecución **horaria** y sin *catchup*. Consta de tres tareas: `evaluate_drift` (`BranchPythonOperator`) ejecuta el análisis de Evidently, empuja las métricas al Pushgateway y decide qué rama ejecutar; `retrain_task` (`PythonOperator`) lanza `docker compose run -rm train` para reentrenar ambos modelos y llama a `POST /reload` para recargarlos en la API **sin reiniciar el servidor**; `skip_task` (`EmptyOperator`) no realiza ninguna acción cuando no hay drift. La tarea de reentrenamiento tiene 1 reintento automático con 2 minutos de espera.

Las alertas de Grafana notifican al equipo vía **Telegram** configurando un *contact point* `telegram-g5` como destino de la política de notificaciones por defecto. Se definen dos reglas de alerta (Figura 5.3):

- **API caída:** se dispara cuando `up{job="core_mp_api"} < 1` se mantiene durante 1 minuto, indicando que Prometheus no puede alcanzar el endpoint de métricas de la API.
- **Latencia P95 alta:** se dispara cuando `histogram_quantile(0.95, ...)` > 0.5 se sostiene durante 2 minutos, señalando una posible degradación del servicio.

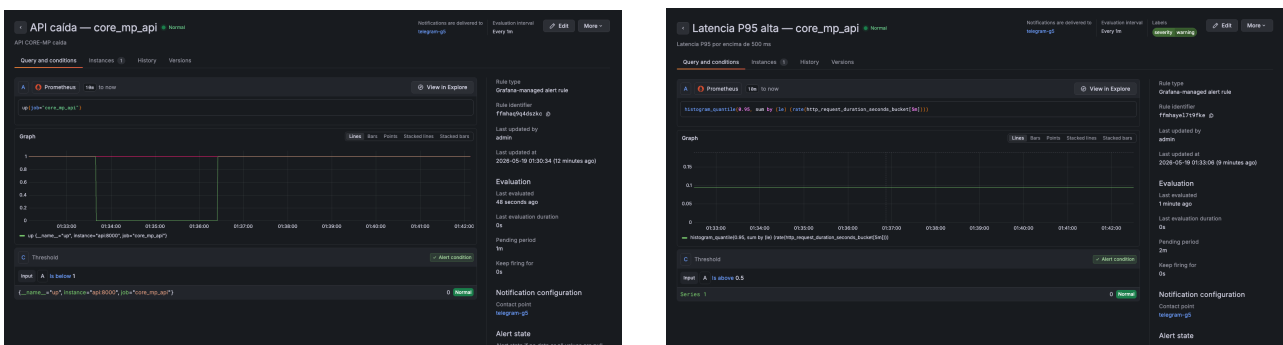


Figure 5.3: Reglas de alerta en Grafana. Izquierda: “API caída” (consulta `up{job="core_mp_api"}`, umbral *below* 1). Derecha: “Latencia P95 alta” (`histogram_quantile(0.95, ...)`, umbral *above* 0.5 s). Ambas notifican vía *contact point* `telegram-g5`.

La Figura 5.4 muestra la pantalla de *Contact points* de Grafana con el canal telegram-g5 activo, usado por 1 política de notificación y 2 reglas de alerta. La validación end-to-end se realizó deteniendo el contenedor de la API (`docker compose stop api`) y verificando la recepción de los mensajes **FIRING** y **RESOLVED** en el grupo de Telegram del equipo (Figura 5.5).

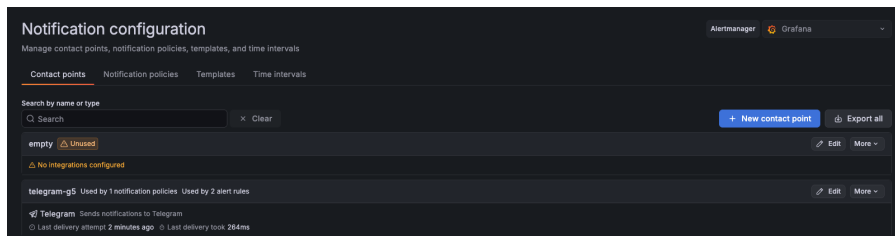


Figure 5.4: Pantalla de *Contact points* de Grafana: el canal telegram-g5 aparece asociado a 1 política de notificación y 2 reglas de alerta, con última entrega hace 2 minutos.



Figure 5.5: Ciclo completo de alerta vía Telegram: mensaje **FIRING** (izquierda) al caer la API y **RESOLVED** (derecha) al recuperarse, 1 min después.

5.6 Ciclo de Vida del Modelo

La Figura 5.6 integra todos los componentes en una visión global del ciclo autónomo del modelo *CORE-MP*.

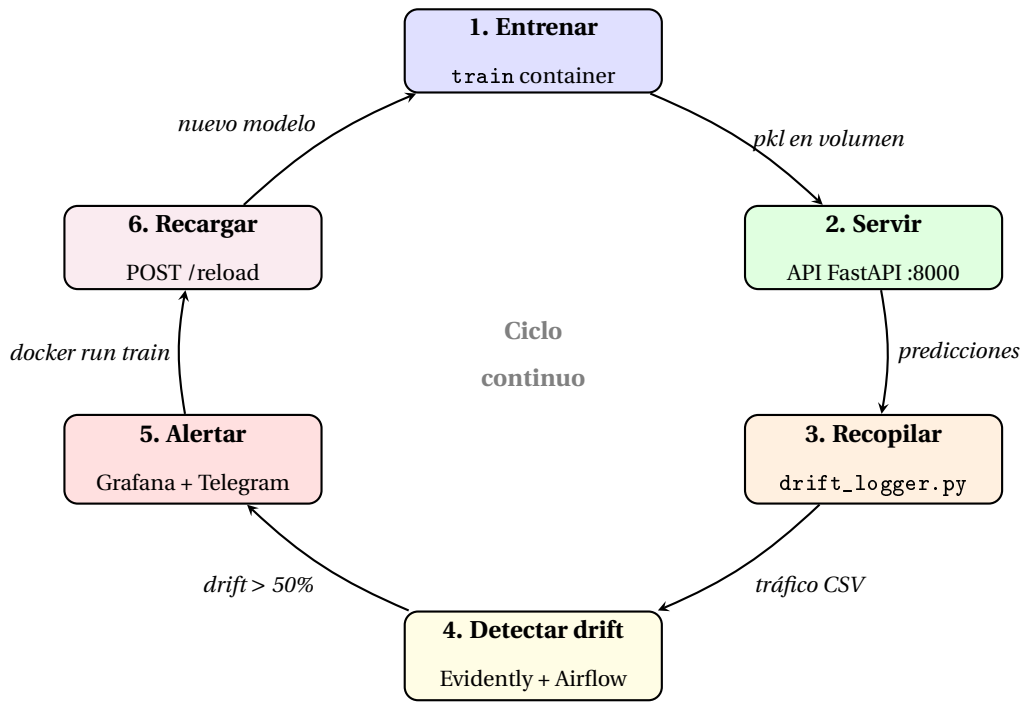


Figure 5.6: Ciclo de vida continuo del modelo CORE-MP: las predicciones alimentan la detección de drift, que activa el reentrenamiento y la recarga sin interrumpir el servicio.

5.7 Conclusiones

La implementación de este hito transforma *CORE-MP* en un sistema MLOps de madurez avanzada. Sus tres aportaciones principales son:

1. **Observabilidad completa:** métricas operativas (latencia P95, tasa de peticiones, errores 5xx) y métricas de calidad del modelo (drift de datos) confluyen en un *dashboard* unificado en Grafana, con alertas proactivas vía Telegram.
2. **Detección y respuesta automática al drift:** Evidently identifica cambios en la distribución de entrada antes de que se manifiesten como errores perceptibles; Airflow cierra el bucle reentrenando el modelo y recargando la API sin interrupción del servicio.
3. **Infraestructura como código:** toda la pila (`docker-compose.yml`, provisioning de Grafana y DAG de Airflow) se define como código, permitiendo reproducir el entorno completo con un único comando.

El proyecto *CORE-MP* ha recorrido de forma íntegra el ciclo de vida de un sistema MLOps: desde el análisis exploratorio del *dataset Cupid's Algorithm* (100.000 parejas, 30 variables) hasta la monitorización continua y el reentrenamiento automático en producción.

El análisis de datos confirmó que la compatibilidad relacional responde a un patrón de **homofilia**, lo que orientó la ingeniería de características hacia diferencias y similitudes entre los rasgos de ambos individuos. Sobre esta base se entrenaron dos modelos interpretables: Regresión Logística para clasificación ($F1 = 0,68$, $ROC-AUC = 0,79$) y Ridge para regresión ($MAE = 16,27$ meses). La elección de modelos interpretables frente a alternativas de mayor capacidad fue deliberada: en el ámbito de las relaciones humanas, la explicabilidad y la robustez ante distribuciones de producción tienen más valor que una ganancia marginal en métricas de validación.

El sistema se desplegó con FastAPI y Docker, serializando el Pipeline completo de scikit-learn para eliminar cualquier *training-serving skew*. La capa de monitorización (Prometheus, Grafana, Evidently, Airflow) cierra un ciclo de *Continuous Training* autónomo: detecta drift cada hora y, cuando supera el umbral del 50%, reentrena y recarga los modelos sin interrumpir el servicio, notificando al equipo vía Telegram.

El principal aprendizaje del proyecto es que **la calidad de un sistema ML no se mide solo por sus métricas de validación, sino por su capacidad de operar de forma fiable y autónoma en producción**. Como trabajo futuro, destacan la incorporación de la tarea de *ranking* (MRR, MAP) a la API, umbrales de drift adaptativos y técnicas de explicabilidad (SHAP) en la respuesta de predicción.